

Enterprise AI Platform — Architecture & Engineering Handbook

Version: 2.0

Date: June 2026

Status: Published

Audience: Staff Engineers, Principal Engineers, Architects, AI Platform Engineers

Table of Contents

Part I — Platform Overview

1. [Executive Summary](#)
2. [Platform Vision](#)
3. [Design Philosophy](#)

Part II — Architecture

4. [System Context](#)
5. [Module Overview](#)
6. [Runtime Architecture](#)

Part III — Core Subsystems

7. [Document Lifecycle](#)
8. [AI Architecture](#)
9. [Semantic Enrichment](#)
10. [GraphRAG](#)
11. [Retrieval Architecture](#)
12. [Prompt Architecture](#)
13. [Model Architecture](#)
14. [Workflow Engine](#)

Part IV — Operations

15. [Explainability](#)
16. [Observability](#)
17. [Security](#)
18. [Testing Strategy](#)
19. [Deployment](#)

Part V — Engineering

20. [Extending the Platform](#)
21. [Performance Considerations](#)
22. [Architectural Trade-offs](#)
23. [Lessons Learned](#)
24. [Future Evolution](#)

Part VI — Reference

25. [Glossary](#)
26. [References](#)

Appendices

- [Appendix A — Architecture Decision Records](#)
- [Appendix B — Diagram Index](#)

- [Appendix C — Table Index](#)
- [Appendix D — Code Example Index](#)

List of Figures

#	Diagram	Section
1	System Context (C4 Level 1)	§4
2	Container Diagram (C4 Level 2)	§4
3	Module Dependencies	§5
4	Component Diagram (C4 Level 3)	§6
5	Document Ingestion Pipeline	§7
6	AI Inference Pipeline	§8
7	Provider SPI Architecture	§8
8	Semantic Enrichment Engine	§9
9	GraphRAG Retrieval Flow	§10
10	Retrieval Orchestration Decision Tree	§11
11	Hybrid Search Fusion	§11
12	Prompt Registry Structure	§12
13	Model Capability Registry	§13
14	Provider Router Decision Flow	§13
15	Workflow Engine State Machine	§14
16	Explainability Metadata Model	§15
17	Observability Architecture	§16
18	Deployment Architecture	§19
19	Testing Pyramid	§18
20	Document Upload Sequence	§7
21	Search Request Sequence	§11
22	AI Inference Sequence	§8
23	GraphRAG Retrieval Sequence	§10
24	Provider Routing Sequence	§13
25	Workflow Execution Sequence	§14

1. Executive Summary

1.1 What Is the Enterprise AI Platform?

Definition: A reusable, modular monolith in Java 21 and Spring Boot 3.3 providing infrastructure for AI-powered enterprise applications — retrieval-augmented generation, semantic enrichment, knowledge graph construction, multi-provider AI orchestration, evaluation, explainability, and production observability.

This is not a finished SaaS product. It is a **foundation** — reusable modules, interfaces, and architectural patterns that support building AI-powered applications on a common core.

1.2 Purpose

Most AI demonstrations are single-file Python scripts calling an LLM API. They lack modularity, testability, observability, and resilience. This platform demonstrates how an experienced engineering team integrates AI into enterprise software: clear boundaries, abstract interfaces, graceful degradation, and full auditability.

Three purposes drive the platform:

- 1. **Reference implementation** — production-grade AI engineering in Java
- 2. **Reusable foundation** — multiple AI applications from a single codebase
- 3. **Portfolio demonstration** — Staff Engineer-level architecture and judgement

1.3 Scope

In Scope	Out of Scope
Multi-provider AI orchestration (Ollama, OpenAI-compatible)	Microservices deployment
RAG with hybrid search (keyword + vector + graph)	Multi-agent AI systems
Semantic enrichment with automatic entity/concept extraction	Model fine-tuning infrastructure
GraphRAG with Neo4j knowledge graph traversal	User management / multi-tenancy
Prompt registry with versioning and categories	Billing / usage tracking
Model capability registry with intelligent routing	Real-time collaboration
Evaluation framework (grounding, faithfulness, hallucination)	
Full explainability metadata on every inference	
Workflow engine for multi-step processes	
Production observability (Micrometer, Prometheus, health checks)	
Graceful degradation for all external dependencies	
157 automated tests across 5 testing layers	

1.4 Target Applications

The platform supports building applications for document intelligence, contract analysis, enterprise search, compliance, financial analysis, and research platforms. The included *Document Intelligence* application is the first reference implementation.

Related ADRs: [ADR-001](#), [ADR-020](#)

2. Platform Vision

2.1 AI as Infrastructure

Core thesis: *AI is infrastructure, not a feature.*

LLMs, embedding models, vector databases, and knowledge graphs are infrastructure components — analogous to databases, message queues, and caches. They belong behind interfaces, selected via configuration, swappable without changing business logic.

A `DocumentService` never calls `Ollama`. It calls `EnrichmentService`, which may be backed by Ollama, OpenAI, or a regex fallback. This distinction runs through every module.

2.2 Domain Independence

The platform core contains no domain-specific logic. No assumptions about legal documents, financial reports, or any industry. Domain customization uses the `DomainConfiguration` SPI — a pluggable interface providing concept definitions, analysis objectives, finding hierarchies, and system instructions.

The Document Intelligence reference application provides default configurations. New domains — contract analysis, financial review, compliance — add `DomainConfiguration` implementations without modifying platform code.

Related ADR: [ADR-017](#)

2.3 Platform vs Application

A critical architectural distinction:

- **Platform modules** (`platform-*`) provide infrastructure — SPIs, orchestration, reusable services
- **Application module** (`platform-api`) wires the platform into a runnable application — REST controllers, templates, configuration

Future applications (Contract Intelligence, Financial Analysis, Compliance) create their own application modules depending on the same platform modules.

```
// Platform SPI – lives in platform-ai
public interface ChatCompletionProvider {
    String providerName();
    boolean isAvailable();
    String complete(String prompt, ModelCapabilities capabilities);
}

// Application wiring – lives in platform-api
@Bean
@ConditionalOnProperty(name = "platform.ai.ollama.base-url")
public ChatCompletionProvider ollamaProvider(AiProviderProperties props) {
    return new OllamaChatProvider(props);
}
```

Related ADR: [ADR-020](#)

3. Design Philosophy

3.1 Eight Governing Principles

Every architectural decision is evaluated against these principles. When a decision conflicts, it is rejected or the principle is amended with explicit rationale.

#	Principle	Rationale	ADR
1	AI is Infrastructure	LLMs, embeddings, and vector stores are abstracted behind interfaces	ADR-003
2	Modular Monolith	Compile-time module boundaries enforce dependency direction	ADR-001
3	Graceful Degradation	Every external dependency is optional	ADR-016
4	Explainability by Default	Every inference produces auditable metadata	ADR-012
5	Documents as Knowledge Source	Knowledge is extracted during ingestion, never manually curated	ADR-007
6	Domain Independence	Platform core contains no domain-specific logic	ADR-017
7	Testing as Documentation	Tests validate behavior; test structure mirrors architecture	ADR-019
8	Production Readiness	Observability, health checks, and configuration are first-class	ADR-015

3.2 Modularity

Module boundaries are enforced at **compile time** through Maven dependency resolution. The dependency graph is directional — higher-level modules depend on lower-level APIs, never the reverse.

```
graph TB
  api[platform-api]
  ai[platform-ai]
  search[platform-search]
  neo4j[platform-neo4j]
  doc[platform-document]
  ws[platform-workspace]
  obs[platform-observability]
  auth[platform-auth]
  audit[platform-audit]

  api --> ai
  api --> search
  api --> doc
  api --> ws
  api --> obs
  api --> auth
  ai --> search
  ai --> audit
  search --> doc
  search --> audit
  doc --> audit
  ws --> doc
```

```
ws --> audit
auth --> audit
```

3.3 Extensibility

Extension points are Service Provider Interfaces (SPIs). The platform defines the contract; implementations provide the behavior.

SPI	Location	Activation
ChatCompletionProvider	platform-ai/api/	@ConditionalOnProperty
EmbeddingProvider	platform-search/api/	@ConditionalOnProperty
GraphSearchProvider	platform-search/api/	@ConditionalOnBean
VectorSearchProvider	platform-search/api/	@ConditionalOnProperty
EnrichmentService	platform-ai/api/	Always active (falls back to regex)
EvaluationService	platform-ai/api/	Always active
DomainConfiguration	platform-ai/api/	@Component
WorkflowEngine	platform-ai/api/	Always active

New implementations activate via `@Component` + `@ConditionalOnProperty` — zero code changes to existing services.

3.4 Testing Philosophy

Tests are executable architecture documentation. A Staff Engineer should understand the platform by reading the test suite. Tests describe behavior, not implementation:

- ✔ "routes openai:gpt-4o to openai provider"
- ✔ "skips unavailable provider and selects available one"
- ✘ "testProviderRouter"
- ✘ "testGetLatest"

Related ADRs: [ADR-020](#), [ADR-019](#)

4. System Context

4.1 C4 Level 1 — System Context

```
C4Context
  title System Context – Enterprise AI Platform

  Person(user, "Engineer", "Uploads documents, runs searches, asks AI questions")

  System(platform, "Enterprise AI Platform", "Java 21 / Spring Boot 3.3 modular monolith")

  System_Ext(postgres, "PostgreSQL", "Relational persistence – documents, users, chunks, audit")
  System_Ext(qdrant, "Qdrant", "Vector similarity search (optional)")
  System_Ext(neo4j, "Neo4j", "Knowledge graph persistence (optional)")
  System_Ext(ollama, "Ollama", "Local LLM inference and embeddings (optional)")
  System_Ext(openai, "OpenAI API", "Cloud LLM inference (optional)")
```

```
Rel(user, platform, "Uses browser on :8080", "HTTPS")
Rel(platform, postgres, "Reads/writes", "JDBC :5433")
Rel(platform, qdrant, "Indexes/searches vectors", "REST :6333")
Rel(platform, neo4j, "Persists/traverses graph", "Bolt :7687")
Rel(platform, ollama, "Inference, embeddings", "REST :11434")
Rel(platform, openai, "Inference", "REST HTTPS")
```

4.2 C4 Level 2 — Container Diagram

```
C4Container
    title Container Diagram – Enterprise AI Platform

    Container(web, "Web Application", "Spring Boot / Thymeleaf", "REST + server-rendered UI")
    Container(db, "PostgreSQL", "pgvector/pg16", "Documents, users, chunks, audit log")
    Container(qdrant_svc, "Qdrant", "Vector DB", "Embedding similarity search")
    Container(neo4j_svc, "Neo4j", "Graph DB", "Knowledge graph traversal")
    Container(llm, "LLM Server", "Ollama / OpenAI", "Inference and embeddings")

    Rel(web, db, "JDBC", "Persistent state")
    Rel(web, qdrant_svc, "REST API", "Vector indexing and search")
    Rel(web, neo4j_svc, "Bolt", "Graph persistence and traversal")
    Rel(web, llm, "HTTP REST", "Chat completion and embeddings")
```

4.3 Technology Stack

Layer	Technology	Version	Notes
Language	Java	21	Virtual threads available
Framework	Spring Boot	3.3.5	Via BOM
Security	Spring Security + Nimbus JOSE	Boot-managed	HS256 JWT, BCrypt-12
Persistence	Spring Data JPA + Hibernate	Boot-managed	PostgreSQL/H2
Vector DB	Qdrant	latest	REST API
Graph DB	Neo4j	5-community	Bolt protocol
LLM Backend	Ollama (local), OpenAI (cloud)	—	HTTP REST
Metrics	Micrometer + Prometheus	Boot-managed	/actuator/prometheus
UI	Thymeleaf + Bootstrap 5	Boot-managed	Server-side rendering
Testing	JUnit 5 + Spring Boot Test + Playwright	—	157 tests
Build	Maven	3.x	9 modules

5. Module Overview

5.1 Module Dependency Graph

```

graph TD
    subgraph "Application Layer"
        api[platform-api<br/>REST + Thymeleaf + Assembly]
    end

    subgraph "Platform Layer"
        ai[platform-ai<br/>RAG Orchestration]
        search[platform-search<br/>Hybrid Search]
        neo4j[platform-neo4j<br/>Knowledge Graph]
        doc[platform-document<br/>Document Lifecycle]
        ws[platform-workspace<br/>Workspace Wizard]
        obs[platform-observability<br/>Metrics + Health]
        auth[platform-auth<br/>JWT Authentication]
    end

    subgraph "Foundation Layer"
        audit[platform-audit<br/>Immutable Audit Log]
    end

    api --> ai
    api --> search
    api --> doc
    api --> ws
    api --> obs
    api --> auth
    api --> neo4j
    ai --> search
    ai --> audit
    search --> doc
    search --> audit
    neo4j --> audit
    doc --> audit
    ws --> doc
    ws --> audit
    auth --> audit
    obs --> audit

```

5.2 Module Responsibilities

platform-audit

Role: Immutable audit logging — leaf module, dependency of all others.

Key API: `AuditService.emit(AuditEventType, AuditSubject, AuditSource)`

Implementation: JDBC-backed `PersistentAuditService` with `CorrelationIdFilter` for X-Correlation-Id injection.

platform-auth

Role: JWT authentication (HS256) with BCrypt-12 and refresh token rotation.

Key API: `AuthFacade` — register, login, refresh, logout, current user.

Implementation: `JwtTokenService` (Nimbus JOSE), `JpaUserDetailsService`, `SecurityConfiguration`.

platform-document

Role: Document lifecycle — creation, versioning, ingestion scheduling.

Key API: `DocumentFacade` — CRUD, `TextExtractionService` — extract text from PDF/DOCX/TXT/HTML.

Implementation: PDFBox 3.0.3, POI 5.4.0, JSoup 1.18.1. Scheduled `DocumentIngestionWorker` polls every 10s.

platform-search

Role: Hybrid search — keyword (JPA/TF), vector (Qdrant), graph (Neo4j).

Key API: `SearchFacade` , `HybridRetrievalService` , `GraphSearchProvider` .

Implementation: Weighted linear fusion ($k \times 0.40 + v \times 0.40 + c \times 0.20$), sentence-aware chunking (1200-char target, 150-char overlap), two-stage reranking.

platform-ai

Role: Heart of the platform — 31 interfaces across RAG orchestration, enrichment, evaluation, prompt registry, model capability registry, provider routing, and workflow engine.

Key API: `AiFacade` , `PromptRegistry` , `ProviderRouter` , `EnrichmentService` , `EvaluationService` , `DomainConfiguration` , `WorkflowEngine` .

platform-neo4j

Role: Graph persistence for enrichment output. Auto-generated during ingestion.

Key API: `GraphEnrichmentService` — persist, traverse, find related documents.

Implementation: Neo4j Java Driver, `@ConditionalOnProperty(name = "platform.neo4j.uri")` , `NodeProvenance` on every element.

platform-workspace

Role: Workspace management with 5-phase wizard.

Key API: `WorkspaceService` — CRUD, phase transitions, document linking, timeline.

platform-observability

Role: Micrometer + Prometheus metrics, health indicators. Reusable across applications.

Key API: `AiMetrics` — timers, counters, gauges for all AI operations.

platform-api

Role: Application assembly — wires all modules into a runnable Spring Boot application.

Key configuration: `SearchInfrastructureConfig` for conditional bean wiring.

6. Runtime Architecture

6.1 C4 Level 3 — Component Diagram

```
C4Component
    title Component Diagram – AI Inference Path

    Container_Boundary(platform, "Enterprise AI Platform") {
        Component(controller, "AiPageController", "Thymeleaf", "Renders AI query form and results")
        Component(aiFacade, "AiFacade", "Java SPI", "Orchestrates full AI pipeline")
        Component(promptReg, "PromptRegistry", "Java SPI", "Versioned prompt templates")
        Component(providerRouter, "ProviderRouter", "Java SPI", "4-tier model selection")
        Component(retrievalOrch, "RetrievalOrchestrator", "Java SPI", "Intent → strategy → search")
        Component(evaluation, "EvaluationService", "Java SPI", "Quality assessment")

        Component(llmProvider, "ChatCompletionProvider", "Java SPI", "LLM backend")
        Component(searchFacade, "SearchFacade", "Java SPI", "Hybrid search entry point")
    }
```

```

}

Rel(controller, aiFacade, "Calls answer()")
Rel(aiFacade, retrievalOrch, "Orchestrates retrieval")
Rel(aiFacade, promptReg, "Resolves prompt template")
Rel(aiFacade, providerRouter, "Routes to provider")
Rel(aiFacade, evaluation, "Evaluates quality")
Rel(aiFacade, llmProvider, "Calls complete()")
Rel(aiFacade, searchFacade, "Executes search")

```

6.2 Startup Sequence

```

sequenceDiagram
    participant Main as main()
    participant Spring as SpringApplication
    participant Scan as Component Scan
    participant Config as @ConfigurationProperties
    participant Conditional as @ConditionalOnProperty
    participant Registry as Registries
    participant Health as Health Indicators
    participant Tomcat as Embedded Tomcat

    Main->>Spring: run(PlatformApiApplication.class)
    Spring->>Scan: scanBasePackages("com.cognitera.platform")
    Scan-->>Spring: @Service, @Component, @Repository beans
    Spring->>Config: Bind application.yml to typed records
    Spring->>Conditional: Evaluate activation conditions
    Conditional-->>Spring: Activate/deactivate provider beans
    Spring->>Registry: Seed PromptRegistry, ModelCapabilityRegistry, WorkflowEngine
    Spring->>Health: Auto-discover HealthIndicator beans
    Spring->>Tomcat: Start on port 8080
    Tomcat-->>Main: Ready

```

[!NOTE] All external dependency beans are gated by `@ConditionalOnProperty`. If `platform.search.qdrant.host` is absent, `QdrantVectorSearchProvider` is never created, and `NoOpVectorSearchProvider` serves as the fallback. The platform starts with only PostgreSQL — all other infrastructure is optional.

6.3 Dependency Injection

Constructor injection exclusively. No `@Autowired` on fields. Every dependency is explicit. Optional dependencies use `ObjectProvider<T>` :

```

@Component
public class DefaultDocumentIngestionProcessor implements DocumentIngestionProcessor {

    private final DocumentFacade documents;
    private final ObjectProvider<EnrichmentHook> enrichmentHook;

    public DefaultDocumentIngestionProcessor(
        DocumentFacade documents,
        ObjectProvider<EnrichmentHook> enrichmentHook) {
        this.documents = documents;
        this.enrichmentHook = enrichmentHook;
    }
}

```

```

    }

    public void ingest(UUID documentId) {
        // ...text extraction and chunking...
        EnrichmentHook hook = enrichmentHook.getIfAvailable();
        if (hook != null) {
            hook.enrich(documentId, title, text);
        }
    }
}

```

Related ADRs: [ADR-002](#), [ADR-003](#), [ADR-016](#)

7. Document Lifecycle

7.1 Ingestion Pipeline

```

sequenceDiagram
    actor User
    participant Controller as DocumentController
    participant Service as DocumentService
    participant Worker as IngestionWorker
    participant Processor as IngestionProcessor
    participant Extraction as TextExtractionService
    participant Enrichment as EnrichmentHook
    participant Chunking as ChunkingStrategy
    participant Embedding as EmbeddingProvider
    participant PostgreSQL
    participant Qdrant
    participant Neo4j

    User->>Controller: POST /api/documents (multipart)
    Controller->>Service: createDocument(file)
    Service->>PostgreSQL: INSERT document + ingestion_job (PENDING)
    Service-->>Controller: Document created

    Note over Worker: @Scheduled every 10s
    Worker->>PostgreSQL: SELECT pending jobs (LIMIT 10)
    loop For each pending job
        Worker->>Processor: ingest(documentId)
        Processor->>Extraction: extractText(type, version)
        Extraction-->>Processor: Raw text

        Processor->>Enrichment: enrich(documentId, text)
        Enrichment->>Neo4j: Persist entities, concepts, relationships

        Processor->>Chunking: chunk(text)
        Chunking-->>Processor: List<DocumentChunk>

        Processor->>Embedding: embedBatch(texts)
        Embedding-->>Processor: List<float[]>

        Processor->>PostgreSQL: INSERT chunks (JPA)
    
```

```
Processor->>Qdrant: PUT points with vectors
Processor->>PostgreSQL: UPDATE ingestion_job (COMPLETED)
end
```

7.2 Chunking Strategy

```
// SentenceAwareChunkingStrategy – splits at sentence boundaries
public class SentenceAwareChunkingStrategy implements ChunkingStrategy {
    private static final int TARGET_CHUNK_SIZE = 1200;
    private static final int MIN_CHUNK_SIZE = 200;
    private static final int CHUNK_OVERLAP = 150;

    @Override
    public List<DocumentChunk> chunk(UUID documentId, int version,
                                     String title, String text) {
        // Split at .!? followed by capital letter or newline
        // Target 1200 chars, minimum 200, 150-char overlap
    }
}
```

7.3 Graceful Degradation in Ingestion

Scenario	Behavior
No embedding config	Keyword-only indexing (no vectors)
No Qdrant	Vectors skipped, keyword search works
No Neo4j	Enrichment runs, graph persistence skipped
No LLM	Regex-based enrichment fallback

Related ADRs: [ADR-007](#), [ADR-010](#), [ADR-016](#)

8. AI Architecture

8.1 AI Inference Pipeline

```
sequenceDiagram
    actor User
    participant Controller as AiPageController
    participant Facade as AiFacade
    participant Service as AiService
    participant Intent as QueryIntentClassifier
    participant Retrieval as RetrievalAugmentationService
    participant Search as SearchFacade
    participant Prompt as PromptRegistry
    participant Router as ProviderRouter
    participant LLM as ChatCompletionProvider
    participant Validation as ClaimValidator
    participant Grounding as GroundingService
    participant Eval as EvaluationService
```

```

User->>Controller: POST /ai (question)
Controller->>Facade: answer(AiRequest)
Facade->>Service: answer()

Service->>Intent: classify(question)
Intent-->>Service: QUERY_ANSWER / INDEX_INSPECTION

Service->>Retrieval: retrieve(request)
Retrieval->>Search: search(query, HYBRID)
Search-->>Retrieval: SearchResultPage
Retrieval-->>Service: RetrievalContext

Service->>Prompt: getLatest("rag-answer")
Prompt-->>Service: PromptTemplate (v1, RETRIEVAL)
Service->>Service: Build and render prompt

Service->>Router: routeChat(request)
Router-->>Service: ChatCompletionProvider
Service->>LLM: complete(prompt, capabilities)
LLM-->>Service: Raw answer text

Service->>Validation: validate(question, answer)
Validation-->>Service: ValidationResult

Service->>Grounding: ground(answer, sources)
Grounding-->>Service: ReasonedAnswer

Service->>Eval: evaluate(question, answer, context)
Eval-->>Service: EvaluationResult

Service-->>Controller: AiResponse(answer, metadata)
Controller-->>User: Rendered HTML with answer + sources

```

8.2 Provider SPI Architecture

```

classDiagram
    class ChatCompletionProvider {
        <<interface>>
        +providerName() String
        +isAvailable() boolean
        +complete(String prompt, ModelCapabilities) String
    }

    class OllamaChatProvider {
        -HttpClient httpClient
        -AiProviderProperties properties
        +complete() String
    }

    class OpenAiChatProvider {
        -HttpClient httpClient
        -AiProviderProperties properties
        +complete() String
    }

```

```

class ProviderRouter {
    <<interface>>
    +routeChat(InferenceRequest) ChatCompletionProvider
    +resolveProviders(CapabilityRequest) List
}

class DefaultProviderRouter {
    -List~ChatCompletionProvider~ providers
    +routeChat() ChatCompletionProvider
}

ChatCompletionProvider <|.. OllamaChatProvider
ChatCompletionProvider <|.. OpenAiChatProvider
ProviderRouter <|.. DefaultProviderRouter
DefaultProviderRouter --> ChatCompletionProvider : selects from

```

8.3 Key API — ChatCompletionProvider

```

/**
 * SPI for AI chat completion backends.
 * Implementations handle the HTTP details of a specific provider's chat API.
 * Activated by @ConditionalOnProperty – business logic never depends on implementations.
 */
public interface ChatCompletionProvider {
    /** Human-readable provider name (e.g. "ollama", "openai"). */
    String providerName();

    /** Returns true if the provider is currently reachable. */
    boolean isAvailable();

    /** Sends a prompt to the model and returns generated text. */
    String complete(String prompt, ModelCapabilities capabilities);
}

```

Related ADRs: [ADR-003](#), [ADR-004](#), [ADR-011](#)

9. Semantic Enrichment

9.1 Enrichment Pipeline

```

flowchart LR
    A[Document Text] --> B{LLM Available?}
    B -->|Yes| C[LLM Extraction<br/>entity-extraction/v1 prompt]
    B -->|No| D[Regex Extraction<br/>ORGANIZATION, PERSON, DATE, MONEY]
    C --> E[EnrichmentContext]
    D --> E
    E --> F[EnrichmentHook]
    F --> G{Neo4j Available?}
    G -->|Yes| H[GraphEnrichmentService.persist]

```

```
G -->|No| I[Skip – enrichment result discarded]
H --> J[Neo4j Graph<br/>with NodeProvenance]
```

9.2 Key API — EnrichmentService

```
/**
 * SPI for semantic enrichment of document text during ingestion.
 * Extracts entities, concepts, and relationships from raw text.
 * Results are automatically persisted to the knowledge graph when available.
 */
public interface EnrichmentService {
    /**
     * Enriches the given text with extracted semantic information.
     * @param documentId the document being enriched
     * @param text the raw text content
     * @return enrichment context containing entities, concepts, and relationships
     */
    EnrichmentContext enrich(String documentId, String text);
}
```

9.3 Provenance

Every node and relationship carries `NodeProvenance` :

```
public record NodeProvenance(
    String sourceDocumentId,    // Which document
    String chunkId,            // Which chunk within document
    Integer chunkOffset,       // Position within chunk
    double extractionConfidence, // 0.0-1.0
    Instant extractionTimestamp, // When extracted
    String extractionModel,     // Which model (or "regex")
    String promptVersion,       // Which prompt template version
    String provider             // Which AI provider
) {}
```

Related ADRs: [ADR-007](#), [ADR-018](#)

10. GraphRAG

10.1 GraphRAG Retrieval Flow

```
flowchart TB
    subgraph Retrieval["Retrieval Pipeline"]
        Q[User Query]
        KW[Keyword Search<br/>JPA/TF scoring]
        VEC[Vector Search<br/>Qdrant cosine]
        GRAPH[Graph Search<br/>Neo4j traversal]
        FUSION[Weighted Linear Fusion]
        RERANK[Two-Stage Reranking]
        CAND[Ranked Candidates]
    end
```

```

Q --> KW
Q --> VEC
Q --> GRAPH
KW --> FUSION
VEC --> FUSION
GRAPH --> FUSION
FUSION --> RERANK
RERANK --> CAND

GRAPH -->|optional| NEO4J[Neo4jGraphSearchAdapter]
NEO4J -->|bridges to| GES[GraphEnrichmentService]

style GRAPH fill:#f9f,stroke:#333,stroke-dasharray: 5 5
style NEO4J fill:#f9f,stroke:#333,stroke-dasharray: 5 5

```

10.2 Fusion Integration

```

sequenceDiagram
    participant Orchestrator as RetrievalOrchestrator
    participant Hybrid as DefaultHybridRetrievalService
    participant Keyword as KeywordSearchProvider
    participant Vector as VectorSearchProvider
    participant Graph as GraphSearchProvider
    participant Neo4j as Neo4jGraphSearchAdapter

    Orchestrator->>Hybrid: retrieve(query)

    par Parallel Retrieval
        Hybrid->>Keyword: search(query)
        Keyword-->>Hybrid: keywordResults (10-20)
        Hybrid->>Vector: search(query)
        Vector-->>Hybrid: vectorResults (10-20)
        Hybrid->>Graph: search(query)
        Graph->>Graph: isAvailable()?
        alt Neo4j Available
            Graph->>Neo4j: traverse(seedIds, 2)
            Neo4j-->>Graph: List<GraphNode>
            Graph-->>Hybrid: graphResults (3-8)
        else Neo4j Unavailable
            Graph-->>Hybrid: empty list
        end
    end

    Hybrid->>Hybrid: merge(keyword, vector, graph)
    Note over Hybrid: keyword×0.40 + vector×0.40<br/>+ confidence×0.20<br/>+ graphBoost (up to 0.15)
    Hybrid-->>Orchestrator: Fused candidates

```

10.3 Key API — GraphSearchProvider

```

/**
 * SPI for graph-based retrieval.

```

```

* Optional – retrieval continues with keyword+vector when unavailable.
* The Neo4jGraphSearchAdapter bridges platform-neo4j to this SPI.
*/
public interface GraphSearchProvider {
    boolean isAvailable();
    List<RetrievalCandidate> search(SearchQuery query);
    List<String> findRelatedDocuments(String documentId, int maxDepth);
}

```

Related ADRs: [ADR-008](#), [ADR-009](#), [ADR-016](#)

11. Retrieval Architecture

11.1 Retrieval Orchestration Decision Tree

```

flowchart TD
    Q[User Query] --> IC[IntentClassifier.classify]
    IC -->|INDEX_INSPECTION| KW[KEYWORD Strategy]
    IC -->|CORPUS_DISCOVERY| KW
    IC -->|QUESTION_ANSWERING| HY[HYBRID Strategy]
    IC -->|WORKSPACE_ANALYSIS| HY
    IC -->|SOURCE_ANALYSIS| HY
    IC -->|DOCUMENT_RESEARCH| SEM[SEMANTIC Strategy]
    IC -->|DOCUMENT_LOOKUP| SEM

    KW --> KWS[KeywordSearchProvider only]
    HY --> HYS[Keyword + Vector + Graph]
    SEM --> SEMS[VectorSearchProvider only]

    KWS --> FUSION[Weighted Linear Fusion]
    HYS --> FUSION
    SEMS --> FUSION
    FUSION --> RERANK[Two-Stage Reranking]
    RERANK --> RESULT[Ranked Candidates]

```

11.2 Hybrid Search Fusion

```

flowchart LR
    subgraph Inputs
        K["Keyword Results<br/>TF score: 0.0-1.0"]
        V["Vector Results<br/>Cosine score: 0.0-1.0"]
        G["Graph Results<br/>Traversal score: 0.0-1.0"]
    end

    subgraph Merge["Merge by Chunk ID (LinkedHashMap dedup)"]
        M["For each unique chunk:"]
    end

    subgraph Scoring["Score Calculation"]
        S["rankingScore = (keyword×0.40 + vector×0.40 + confidence×0.20)<br/>× docTypeWeight<br/>+ graphBoost (max 0.15)<br/>clamped to [0, 1]"]
    end

```

```
K --> Merge
V --> Merge
G --> Merge
Merge --> Scoring
Scoring --> R[Reranking]
R --> C[Final Candidates]
```

11.3 Key API — RetrievalOrchestrator

```
/**
 * Top-level orchestrator for the retrieval pipeline.
 * Intent → Strategy → Retrieval → Fusion → Reranking → Result.
 */
public interface RetrievalOrchestrator {
    RetrievalOrchestrationResult orchestrate(AiRequest request);
}
```

Related ADRs: [ADR-011](#), [ADR-008](#)

12. Prompt Architecture

12.1 Prompt Registry Structure

```
classDiagram
    class PromptTemplate {
        -String id
        -int version
        -Category category
        -String description
        -String template
        -List~String~ variables
        -String expectedOutputType
        -List~String~ supportedModels
        -double recommendedTemperature
        -List~Example~ examples
        -Map~String,String~ metadata
        +getQualifiedId() String
        +render(Map~String,String~) String
    }

    class Category {
        <<enumeration>>
        RETRIEVAL
        SUMMARIZATION
        EXTRACTION
        CLASSIFICATION
        EVALUATION
        REASONING
        WORKFLOW
        GRAPH
        SEARCH
    }
```

```

SYSTEM
}

class Example {
    +Map~String,String~ input
    +String expectedOutput
}

class PromptRegistry {
    <<interface>>
    +get(String qualifiedId) Optional
    +getLatest(String promptId) Optional
    +listPromptIds() List
    +getVersions(String promptId) List
    +findByCategory(Category) List
    +register(PromptTemplate) void
}

PromptTemplate --> Category
PromptTemplate --> Example
PromptRegistry --> PromptTemplate

```

12.2 Key API — PromptRegistry

```

/**
 * Registry of versioned prompt templates.
 * Enables prompt reproducibility, auditing, and regression testing.
 */
public interface PromptRegistry {
    /** Returns prompt by qualified ID (e.g., "rag-answer/v1"). */
    Optional<PromptTemplate> get(String qualifiedId);

    /** Returns the latest version of a prompt. */
    Optional<PromptTemplate> getLatest(String promptId);

    /** Lists all registered prompt IDs. */
    List<String> listPromptIds();

    /** Returns prompts in a given category. */
    List<PromptTemplate> findByCategory(PromptTemplate.Category category);

    /** Registers or updates a prompt template. */
    void register(PromptTemplate template);
}

```

Related ADR: [ADR-005](#)

13. Model Architecture

13.1 Model Capability Registry

```

classDiagram
    class ModelCapability {
        +String modelName
        +String provider
        +boolean supportsStreaming
        +boolean supportsVision
        +boolean supportsJson
        +boolean supportsToolCalling
        +boolean supportsEmbeddings
        +boolean supportsReasoning
        +boolean supportsStructuredOutput
        +int maxContextWindow
        +int maxOutputTokens
        +int estimatedLatencyMs
        +double estimatedCostPer1kTokens
        +double recommendedTemperature
        +List~String~ preferredUseCases
        +List~String~ tags
        +supports(CapabilityRequest) boolean
    }

    class CapabilityRequest {
        <<enumeration>>
        CHAT
        STREAMING
        VISION
        JSON_OUTPUT
        TOOL_CALLING
        EMBEDDING
        REASONING
        STRUCTURED_OUTPUT
    }

    class ModelCapabilityRegistry {
        <<interface>>
        +get(String modelName) Optional
        +findByProvider(String) List
        +findByCapability(CapabilityRequest) List
        +findByProviderAndCapability(String, CapabilityRequest) List
        +listAll() List
    }

    ModelCapability --> CapabilityRequest
    ModelCapabilityRegistry --> ModelCapability

```

13.2 Provider Router Decision Flow

```

flowchart TD
    REQ[InferenceRequest] --> T1{Model prefix?<br/>e.g., openai:gpt-4o}
    T1 -->|Yes| P1[Route to matching provider]
    T1 -->|No| T2{Capability requested?<br/>e.g., STREAMING}
    T2 -->|Yes| P2[Query ModelCapabilityRegistry<br/>→ route to capable provider]
    T2 -->|No| T3{Preferred provider set?}

```

```

T3 -->|Yes, available| P3[Route to preferred provider]
T3 -->|No or unavailable| T4{Any provider available?}
T4 -->|Yes| P4[First available provider]
T4 -->|No| ERR[throw IllegalStateException]

P1 --> DONE[Return ChatCompletionProvider]
P2 --> DONE
P3 --> DONE
P4 --> DONE

```

13.3 Key API — ProviderRouter

```

/**
 * Routes inference requests to the appropriate provider.
 * Strategy: model prefix → capability match → preferred provider → first available.
 */
public interface ProviderRouter {
    ChatCompletionProvider routeChat(InferenceRequest request);
    List<ChatCompletionProvider> resolveProviders(CapabilityRequest capability);
    List<String> listAvailableModels();
}

```

Related ADRs: [ADR-004](#), [ADR-006](#)

14. Workflow Engine

14.1 Workflow Engine State Machine

```

stateDiagram-v2
    [*] --> SETUP: start("document-intelligence")
    SETUP --> INGESTION: advance()
    INGESTION --> ANALYSIS: advance()
    ANALYSIS --> REVIEW: advance()
    REVIEW --> COMPLETE: advance()
    COMPLETE --> COMPLETE: advance() → COMPLETED status

    INGESTION --> SETUP: previous()
    ANALYSIS --> INGESTION: previous()
    REVIEW --> ANALYSIS: previous()
    COMPLETE --> REVIEW: previous()

    state SETUP {
        [*] --> SetupActive
    }
    state COMPLETE {
        CompleteActive --> [*]: advance()
    }

```

14.2 Key API — WorkflowEngine

```

/**
 * Reusable workflow engine for configurable multi-step processes.
 */
public interface WorkflowEngine {
    WorkflowInstance start(String workflowDefinitionId, Map<String, Object> context);
    WorkflowInstance advance(String instanceId);
    WorkflowInstance previous(String instanceId);
    Optional<WorkflowInstance> findInstance(String instanceId);
    List<WorkflowInstance> listActive();

    record WorkflowStep(String id, String name, String description,
                        String handlerType, Map<String, Object> config,
                        List<String> nextSteps) {}

    record WorkflowDefinition(String id, String name, String description,
                             List<WorkflowStep> steps, String initialStep) {}

    record WorkflowInstance(String id, String workflowDefinitionId, String currentStep,
                           WorkflowStatus status, Map<String, Object> context,
                           Instant startedAt, Instant updatedAt) {}
}

```

Related ADR: [ADR-014](#)

15. Explainability

15.1 Explainability Metadata Model

```

classDiagram
    class RetrievalOrchestrationResult {
        +String intent
        +String selectedStrategy
        +String providerName
        +String promptTemplateId
        +int promptTemplateVersion
        +String modelName
        +Instant retrievalStartedAt
        +Instant retrievalCompletedAt
        +int keywordResultCount
        +int vectorResultCount
        +int graphNodeCount
        +int totalChunkCount
        +String fusionMethod
        +boolean rerankingApplied
        +String rerankingProvider
        +Map~String,Object~ evaluationScores
        +List~String~ traceLog
        +explain() String
    }

    class InferenceMetadata {
        +String provider
        +String model
    }

```

```

+Instant requestedAt
+Instant completedAt
+String correlationId
+String requestId
+String promptTemplateVersion
+String retrievalStrategy
+List~String~ referencedChunkIds
+double confidenceScore
}

```

Every AI response carries both `RetrievalOrchestrationResult` (retrieval decisions) and `InferenceMetadata` (inference execution). The `explain()` method produces a human-readable summary:

```

Intent: QUESTION_ANSWERING | Strategy: HYBRID | Provider: ollama | Model: qwen2.5:14b
Prompt: rag-answer/v1 | Sources: 15 | Fusion: weighted-linear-fusion
Reranking: yes (ollama-cross-encoder) | Duration: 245ms

```

Related ADR: [ADR-012](#)

16. Observability

16.1 Observability Architecture

```

flowchart TB
    subgraph Platform["Enterprise AI Platform"]
        AI[AI Operations<br/>Inference, Embedding, Retrieval]
        ING[Document Operations<br/>Ingestion, Enrichment]
        SYS[System Operations<br/>Health, Availability]
    end

    subgraph Metrics["AiMetrics"]
        INF_TIMER[ai.inference.duration<br/>Timer + Histogram]
        EMB_TIMER[ai.embedding.duration<br/>Timer]
        RET_TIMER[ai.retrieval.duration<br/>Timer]
        GRA_TIMER[ai.graph.retrieval.duration<br/>Timer]
        ENR_TIMER[ai.enrichment.duration<br/>Timer]
        PRO_GAUGE[ai.provider.available<br/>Gauge 0/1]
        EVAL_SUM[ai.evaluation.*<br/>Summary]
    end

    subgraph Export["Export"]
        PROM[Prometheus<br/>/actuator/prometheus]
        HEALTH[Health Checks<br/>/actuator/health]
    end

    AI --> INF_TIMER
    AI --> RET_TIMER
    AI --> GRA_TIMER
    AI --> PRO_GAUGE
    ING --> EMB_TIMER
    ING --> ENR_TIMER
    SYS --> PRO_GAUGE

```

```
Metrics --> PROM
Metrics --> HEALTH

PROM --> GRAFANA[Grafana<br/>Dashboards]
HEALTH --> ALERTING[Alerting<br/>Rules]
```

Related ADR: [ADR-015](#)

17. Security

- **Authentication:** JWT (HS256) via Nimbus JOSE. BCrypt-12 password hashing. Refresh tokens SHA-256 hashed before storage. Token rotation on refresh.
- **Authorization:** Role-based (USER, ANALYST, ADMIN). Stateless API auth (Bearer token) + session-based form login.
- **Provider Isolation:** API keys stored in `application.yml` with `${ENV_VAR}` substitution. Never hardcoded.
- **Prompt Safety:** User input injected as `{{question}}` variable into vetted prompt templates. HTML-escaping in controller responses prevents XSS.

18. Testing Strategy

18.1 Testing Pyramid

```
graph TB
    subgraph "Playwright (UI) – 12 tests"
        PW["mvn verify -Pui-tests<br/>Browser user journeys"]
    end
    subgraph "Architecture + Workflow – 19 tests"
        ARCH["mvn verify<br/>End-to-end behavioral validation"]
    end
    subgraph "Integration – 91 tests"
        INT["mvn verify<br/>Subsystem interaction with Spring context"]
    end
    subgraph "Unit – 44 tests"
        UNIT["mvn test<br/>Isolated component behavior"]
    end

    PW --- ARCH --- INT --- UNIT
```

Layer	Count	Command	Purpose
Unit	44	mvn test	Isolated component behavior
Integration	91	mvn verify	Subsystem interaction with Spring context
Architecture	10	mvn verify	End-to-end behavioral validation
Resilience	9	mvn verify	Graceful degradation scenarios
Contract	5	mvn verify	SPI stability protection
Playwright	12	mvn verify -Pui-tests	Browser user journeys

Related ADR: [ADR-019](#)

19. Deployment

19.1 Deployment Architecture

```
C4Deployment
    title Deployment – Enterprise AI Platform

    Deployment_Node(dev, "Development Machine", "Java 21, Maven") {
        Container(app, "Spring Boot Application", "platform-api", "Port 8080")
    }

    Deployment_Node(db_node, "PostgreSQL Server", "Docker") {
        Container(db, "PostgreSQL", "pgvector/pg16", "Port 5433")
    }

    Deployment_Node(qdrant_node, "Qdrant Server", "Docker") {
        Container(qdrant, "Qdrant", "latest", "Ports 6333/6334")
    }

    Deployment_Node(neo4j_node, "Neo4j Server", "Docker – graph profile") {
        Container(neo4j, "Neo4j", "5-community", "Ports 7474/7687")
    }

    Deployment_Node(llm_node, "LLM Server", "Optional") {
        Container(ollama, "Ollama", "latest", "Port 11434")
    }

    Rel(app, db, "JDBC", "TCP :5433")
    Rel(app, qdrant, "REST", "TCP :6333")
    Rel(app, neo4j, "Bolt", "TCP :7687")
    Rel(app, ollama, "REST", "TCP :11434")
```

```
# Start infrastructure
docker compose up -d                                # PostgreSQL + Qdrant
docker compose --profile graph up -d                 # + Neo4j

# Start the platform
mvn spring-boot:run -pl platform-api
```

Related ADRs: [ADR-001](#), [ADR-002](#)

20. Extending the Platform

Adding an LLM Provider

```
// 1. Implement the SPI
@Component
@ConditionalOnProperty(name = "platform.ai.anthropic.api-key")
public class AnthropicChatProvider implements ChatCompletionProvider {
    @Override public String providerName() { return "anthropic"; }
    @Override public boolean isAvailable() { return true; }
```

```

@Override public String complete(String prompt, ModelCapabilities caps) {
    // HTTP call to Anthropic Messages API
}

// 2. Register model capabilities
registry.register(new ModelCapability("claude-3.5-sonnet", "anthropic",
    true, true, true, true, false, false, true,
    200000, 8192, 800, 3.0, 0.7,
    List.of("general", "analysis", "code"),
    List.of("cloud", "frontier")));

// 3. Configure in application.yml
// platform.ai.anthropic.api-key: ${ANTHROPIC_API_KEY}
// ProviderRouter automatically discovers it via List<ChatCompletionProvider>

```

Adding a Domain Configuration

```

@Component
public class ContractIntelligenceDomain implements DomainConfiguration {
    @Override public String domainId() { return "contract-intelligence"; }
    @Override public List<ConceptDefinition> concepts() {
        return List.of(
            new ConceptDefinition("OBLIGATION",
                List.of("obligation", "shall", "must", "required"),
                List.of(), List.of("TERM", "PARTY")),
            new ConceptDefinition("TERMINATION",
                List.of("termination", "cancel", "end", "cease"),
                List.of(), List.of("NOTICE_PERIOD")));
    }
    // ... other methods
}

```

21. Performance Considerations

- **Chunking:** 1200-char target, sentence-boundary-aware, 150-char overlap
- **Embedding:** Sequential embedBatch (loops — parallelizable via virtual threads)
- **Retrieval:** O(n) weighted fusion, 2-hop max graph traversal
- **LLM Inference:** Configurable timeout (default 120s), no streaming
- **Ingestion:** Scheduled worker polls every 10s, processes top 10 PENDING jobs
- **Caching:** No caching layer — each retrieval re-executes

22. Architectural Trade-offs

Decision	Chosen	Alternative	Rationale
Architecture	Modular Monolith	Microservices	Compile-time boundaries sufficient; future extraction possible
Graph DB	Neo4j property graph	RDF triple store	Cypher > SPARQL for traversal queries

Vector DB	Qdrant	pgvector	Purpose-built for vector search; better performance at scale
Provider selection	Provider Router SPI	Hardcoded if/else	Zero-code provider addition
Prompt management	Versioned registry	Inline strings	Audit trails, regression testing
Framework	Spring Boot 3.3	Quarkus	Larger ecosystem, enterprise familiarity
UI testing	Playwright	Selenium	Modern API, auto-waits, parallel execution
Fusion	Weighted linear	Reciprocal Rank Fusion	Simpler, explainable, rank data not always available

Related ADRs: See individual ADRs in Appendix A for detailed rationale.

23. Lessons Learned

- Domain-specific code is easier to remove when interfaces are clean.** The original German tenancy law content (55 BGB references, 25+ German keywords) was removed in Phase 1 without architectural changes — the interfaces had clean separation.
- Unused dependencies accumulate silently.** `spring-ai-ollama` (5 JARs, ~2MB) was on the classpath for months with zero imports. Regular `mvn dependency:analyze` would have caught this.
- Bean wiring tests are critical.** The entire AI pipeline (AiService + 12 collaborators) compiled and passed all tests but was never injected. The AiPageController only showed retrieval results without generating LLM answers. Wiring tests caught this during the v1.0 audit.
- GraphRAG was documented but not implemented.** The architecture manual described Neo4j participating in retrieval for months before the code matched. The `Neo4jGraphSearchAdapter` bridged the gap in Phase 3.
- Prompt versioning proved essential.** What started as a simple registry became the foundation for deterministic behavior validation and audit trails. Every inference now records exactly which prompt was used.
- Tests as documentation works.** Evolving from 128 wiring-verification tests to 157 behavioral tests made the test suite readable by architects without reading the implementation.

24. Future Evolution

Status	Item
✅ Implemented	Modular monolith, multi-provider AI, prompt registry, model registry, semantic enrichment, GraphRAG, retrieval orchestration, evaluation, explainability, workflow engine, observability, graceful degradation, DomainConfiguration SPI, 157 tests
⬢ Partial	DomainConfiguration: SPI exists but 8 services embed configs; AiMetrics: component exists but not fully wired; WorkflowEngine: no PhaseHandler implementations; GraphRAG: simple entity-name matching
🔮 Future	Dynamic model capability discovery, Resilience4j integration, OpenTelemetry, streaming LLM responses, prompt A/B testing, automated evaluation regression, Spring Modulith verification, GraalVM native image

25. Glossary

Term	Definition
Capability	A feature an AI model supports (streaming, vision, JSON mode, embeddings, tool calling)
ChatCompletionProvider	SPI for LLM backends — the platform's abstraction over Ollama, OpenAI, and future providers
Chunk	A segment of document text (~1200 chars) stored and indexed for retrieval
Chunking	The process of splitting document text into overlapping, searchable segments
Citation Coverage	Fraction of AI answer claims that cite a retrieved source
Context Window	Maximum tokens a model can process in a single request
DomainConfiguration	SPI providing domain-specific rules — concepts, objectives, instructions
Embedding	A 768-dimensional vector representation of text, generated by an embedding model
Enrichment	Automatic extraction of entities, concepts, and relationships from documents
Evaluation	Automated quality assessment of AI-generated answers
Explainability	Metadata describing how an inference was produced (provider, model, prompt, strategy)
Faithfulness	How factually accurate an AI answer is relative to its source documents
Fusion	Combining retrieval results from multiple sources (keyword, vector, graph) into a ranked list
GraphRAG	Retrieval-augmented generation enhanced with knowledge graph traversal
Grounding	The degree to which an AI answer is supported by retrieved evidence
Hallucination	AI-generated text not supported by source documents
Hybrid Search	Retrieval combining keyword, vector, and optionally graph search
Inference	The process of an LLM generating text from a prompt
Intent	The classified purpose of a user query (question answering, document lookup, etc.)
Knowledge Graph	A network of entities, concepts, and relationships extracted from documents
LLM	Large Language Model — an AI model trained to generate text
Model Capability Registry	Searchable database of model features, limits, and performance estimates
Modular Monolith	Single deployable with compile-time module boundaries
NodeProvenance	Metadata linking a graph element to its source document and extraction method
Observability	The ability to understand system state through metrics, logs, and health checks
Playwright	Browser automation framework for end-to-end UI testing

Prompt	A structured text template sent to an LLM to guide its response
Prompt Engineering	The practice of designing prompts to produce desired LLM outputs
Prompt Registry	Versioned store of prompt templates with categories and metadata
Provider	An AI backend (Ollama, OpenAI) implementing a platform SPI
Provider Router	Component selecting which provider to use for an inference request
Provenance	Metadata linking data to its origin — source document, extraction method, timestamp
Qdrant	Vector database for high-dimensional similarity search
RAG	Retrieval-Augmented Generation — grounding LLM responses in retrieved documents
Registry	A store of known entities (prompts, models, capabilities) with query capabilities
Reranking	Reordering retrieval candidates to improve relevance (two-stage: sort + LLM cross-encoder)
Retrieval Strategy	Selection of which retrievers to use (keyword, vector, graph) based on query intent
Semantic Enrichment	Automatic extraction of structured knowledge (entities, concepts, relationships) from text
SPI	Service Provider Interface — a pluggable contract for extending the platform
Streaming	Real-time token-by-token LLM output (not yet implemented)
Structured Output	LLM responses constrained to a specific format (JSON, function calls)
Token	A unit of text processed by an LLM (~0.75 words in English)
Tracing	Following a request through all platform subsystems using a correlation ID
Vector	A numerical representation of text meaning, used for semantic similarity search
Vector Database	Database optimized for high-dimensional similarity queries
Workflow	A configurable multi-step process with defined state transitions
Workflow Engine	Reusable component for executing workflows

26. References

Internal Documentation

- [TESTING.md](#) — Testing philosophy, layers, execution guide
- [README.md](#) — Project overview and quickstart
- [adr/INDEX.md](#) — Architecture Decision Record index

Engineering References

- **Spring Boot 3.3 Reference** — Framework documentation for the platform's runtime
- **Spring Modulith** — Runtime module verification (future integration candidate)
- **C4 Model (Simon Brown)** — Architecture visualization approach used throughout this handbook
- **Mermaid** — Diagram-as-code tool used for all diagrams in this handbook
- **Martin Fowler — Patterns of Enterprise Application Architecture** — Foundational patterns for modular design

AI & Retrieval References

- **RAG (Lewis et al., 2020)** — "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks" — foundational paper
- **GraphRAG (Microsoft Research, 2024)** — "From Local to Global: A Graph RAG Approach to Query-Focused Summarization"
- **Neo4j Graph Data Science** — Algorithms for centrality, community detection, and graph embeddings
- **Qdrant Documentation** — Vector search with payload filtering

Operations References

- **Micrometer Documentation** — Metrics instrumentation for JVM applications
- **Prometheus Documentation** — Metrics collection and alerting
- **OpenTelemetry** — Distributed tracing standard (future integration)
- **Playwright Documentation** — Cross-browser automation for end-to-end testing

Appendix A — Architecture Decision Records

ADR Index

ADR	Title	Status	Category	Summary
001	Modular Monolith	Accepted	Architecture	9 Maven modules, compile-time boundaries
002	Spring Boot 3.3	Accepted	Technology	Java 21, virtual threads, rich ecosystem
003	Provider SPI	Accepted	AI Infra	ChatCompletion, Embedding, Reranking SPIs
004	Provider Router	Accepted	AI Infra	4-tier selection: prefix→capability→preferred→fallback
005	Prompt Registry	Accepted	AI Infra	Versioned prompts, 6 categories, renderable
006	Model Capability Registry	Accepted	AI Infra	6 models, 8 capability dimensions
007	Semantic Enrichment	Accepted	Knowledge	Automatic entity/concept extraction during ingestion
008	GraphRAG	Accepted	Knowledge	3-source fusion: keyword+vector+graph
009	Neo4j	Accepted	Knowledge	Graph persistence for enrichment output
010	Qdrant	Accepted	Knowledge	Dedicated vector database
011	Retrieval Orchestration	Accepted	Knowledge	Intent-driven strategy selection
012	Explainability	Accepted	Quality	Metadata on every inference
013	Evaluation Engine	Accepted	Quality	Grounding, faithfulness, hallucination scores
014	Workflow Engine	Accepted	Extensibility	2 pre-registered workflows
015	AI Observability	Accepted	Operations	Micrometer timers, counters, gauges
016	Graceful Degradation	Accepted	Operations	Conditional beans, NoOp fallbacks
017	DomainConfiguration	Accepted	Extensibility	Pluggable domain rules
018	Provenance Graph	Accepted	Knowledge	Auditable graph nodes and relationships

019	Testing Strategy	Accepted	Quality	157 tests across 5 layers
020	Design Philosophy	Accepted	Governance	8 governing principles

ADR-1

ADR-001 — Modular Monolith Architecture

STATUS

Accepted. Implemented in the Maven module structure at `pom.xml`.

CONTEXT

The Enterprise AI Platform must support multiple AI-powered applications (document intelligence, contract analysis, financial review, compliance, etc.) while remaining deployable as a single process. The platform must demonstrate clean separation of concerns without the operational complexity of microservices.

DECISION

Adopt a **Modular Monolith** architecture with 9 Maven modules:

<code>platform-audit</code>	– Immutable audit log, correlation IDs
<code>platform-auth</code>	– JWT authentication, refresh token rotation
<code>platform-document</code>	– Document lifecycle, ingestion pipeline
<code>platform-search</code>	– Hybrid search (keyword + vector + graph)
<code>platform-ai</code>	– RAG orchestration, enrichment, evaluation, registry
<code>platform-neo4j</code>	– Auto-generated knowledge graph with provenance
<code>platform-workspace</code>	– Multi-phase workflow wizard
<code>platform-observability</code>	– Micrometer metrics, health indicators
<code>platform-api</code>	– REST + Thymeleaf controllers, assembly

Module boundaries follow **dependency inversion**: higher-level modules depend on lower-level APIs, never the reverse. `platform-api` is the assembly module that wires everything together.

ALTERNATIVES CONSIDERED

- **Microservices**: Rejected. The platform demonstrates architecture without distributed systems complexity. Microservices add network boundaries, eventual consistency challenges, and deployment overhead that don't benefit a reference implementation.
- **Single JAR without modules**: Rejected. A flat structure would not enforce dependency direction or demonstrate separation of concerns.
- **OSGi modules**: Rejected. Adds runtime modularity complexity. Maven's compile-time enforcement is sufficient.

CONSEQUENCES

- **Clear dependency direction**: `api` → `ai` → `search` → `document` → `audit` (audit is a leaf dependency)
- **Compile-time enforcement**: Modules cannot accidentally depend on each other
- **Single deployable**: One `platform-api` Spring Boot application with all modules on the classpath
- **Test isolation**: Each module can be tested independently

TRADE-OFFS

- Module boundaries are enforced only at compile time, not runtime
- Adding a new module requires POM maintenance
- Some modules (audit) are cross-cutting dependencies of many others

FUTURE EVOLUTION

- Modules may split if responsibilities grow too large (e.g., `platform-ai` could become `platform-ai-core` + `platform-ai-providers`)
- Spring Modulith could be introduced for runtime module verification
- The architecture deliberately supports future extraction of modules into microservices if needed

See also: [\[\[ADR-002\]\]](#), [\[\[ADR-019\]\]](#), [\[\[ADR-020\]\]](#)

ADR-2

ADR-002 — Spring Boot 3.3 as Application Framework

STATUS

Accepted. Implemented in `pom.xml` at `spring-boot-starter-parent:3.3.5`.

CONTEXT

The platform must be recognizable to experienced Java engineers, leverage mature ecosystems for security, persistence, and observability, and remain maintainable by teams familiar with enterprise Java.

DECISION

Use **Spring Boot 3.3** with Java 21 as the application framework. All modules depend on `spring-boot-starter-parent` BOM for version management. Key Spring integrations:

- `spring-boot-starter-web` — REST controllers
- `spring-boot-starter-security` — JWT authentication via Nimbus JOSE
- `spring-boot-starter-data-jpa` — PostgreSQL/H2 persistence
- `spring-boot-starter-actuator` — Health endpoints, metrics
- `spring-boot-starter-thymeleaf` — Server-rendered UI pages
- `spring-boot-starter-validation` — Request validation
- `@ConfigurationProperties` — Type-safe configuration binding
- `@ConditionalOnProperty` / `@ConditionalOnBean` — Provider activation

ALTERNATIVES CONSIDERED

- **Quarkus**: Rejected. Smaller ecosystem, less familiar to enterprise teams, fewer library integrations.
- **Micronaut**: Rejected. Strong compile-time DI but smaller community and fewer reference architectures.
- **Plain Java with manual DI**: Rejected. Would require reimplementing what Spring provides (security, JPA, scheduling, Actuator, property binding).
- **Spring Boot 2.x**: Rejected. Java 21 virtual threads require Spring Boot 3.x.

CONSEQUENCES

- **Rich ecosystem**: Spring Security, Spring Data JPA, Spring Actuator all integrated out of the box
- **Virtual threads**: Java 21 + Spring Boot 3.3 enable `spring.threads.virtual.enabled`
- **Conditional beans**: `@ConditionalOnProperty` enables graceful degradation for optional infrastructure
- **Configuration**: `@ConfigurationProperties` with `platform.*` prefix ensures consistent, type-safe config

TRADE-OFFS

- Startup time is slower than compile-time DI frameworks
- Spring's "magic" can obscure wiring for newcomers (mitigated by explicit `@Bean` definitions in `SearchInfrastructureConfig`)
- Memory footprint is larger than minimalist frameworks

FUTURE EVOLUTION

- Spring Modulith could add runtime module verification
- Spring AI could replace custom provider HTTP clients when mature
- GraalVM native image compilation could reduce startup time for serverless deployments

See also: [\[\[ADR-001\]\]](#), [\[\[ADR-003\]\]](#), [\[\[ADR-015\]\]](#)

ADR-3

ADR-003 — Provider Abstraction via Service Provider Interface (SPI)

STATUS

Accepted. Implemented in `platform-ai/src/main/java/com/cogniterra/platform/ai/api/ChatCompletionProvider.java` and related interfaces.

CONTEXT

AI backends (Ollama, OpenAI, Anthropic, Gemini) expose different APIs, authentication schemes, and request/response formats. Business logic must not depend on any specific provider implementation. Adding a new provider should require zero changes to orchestration code.

DECISION

Define **SPI interfaces** for every AI capability. Implementations are conditionally activated based on configuration:

Interface	Implementations	Activation
ChatCompletionProvider	OllamaChatProvider, OpenAiChatProvider	@ConditionalOnProperty
EmbeddingProvider	OllamaEmbeddingProvider	@ConditionalOnProperty
RerankingProvider	OllamaRerankingProvider	@ConditionalOnProperty
VectorSearchProvider	QdrantVectorSearchProvider, NoOpVectorSearchProvider	Conditional on host
GraphSearchProvider	Neo4jGraphSearchAdapter, NoOpGraphSearchProvider	Conditional on Neo4j
EvaluationService	DefaultEvaluationService	Always active

The `ProviderRouter` (`DefaultProviderRouter`) selects a provider using a 4-tier strategy:

- 1. Model name prefix (`openai:gpt-4o` → `openai`)
- 2. Capability match (streaming → capable provider)
- 3. Preferred provider
- 4. First available fallback

Business services depend on interfaces, never on concrete implementations.

ALTERNATIVES CONSIDERED

- **Hardcoded provider selection in business logic:** Rejected. Would couple orchestration to specific providers and make adding new providers expensive.
- **Spring AI abstraction:** Rejected. At implementation time, Spring AI 1.0.0 provided limited control over provider selection and lacked the capability registry concept. The dependency was removed during build audit.
- **Factory pattern without SPI:** Rejected. An SPI with conditional beans is more idiomatic in Spring and supports auto-discovery.

CONSEQUENCES

- **Provider-agnostic business logic:** `AIService` depends on `ChatCompletionProvider` , not Ollama or OpenAI
- **Conditional activation:** Providers activate only when their configuration is present
- **No-code provider addition:** Implement `ChatCompletionProvider` + `@Component` + `@ConditionalOnProperty`
- **Graceful degradation:** When no provider is available, `ProviderRouter` throws a clear exception

TRADE-OFFS

- Each provider implements its own HTTP client (no shared HTTP infrastructure)
- Provider implementations must handle their own error translation
- Capability discovery is static (seeded in `DefaultModelCapabilityRegistry`) rather than dynamic

FUTURE EVOLUTION

- Dynamic capability discovery via provider API introspection
- Shared HTTP client infrastructure with configurable retry/timeout
- Provider cost and latency metadata for intelligent routing

See also: [\[\[ADR-004\]\]](#), [\[\[ADR-006\]\]](#), [\[\[ADR-016\]\]](#)

ADR-004 — Provider Router for Intelligent Model Selection

STATUS

Accepted. Implemented in `platform-`

`ai/src/main/java/com/cogniterra/platform/ai/application/DefaultProviderRouter.java` .

CONTEXT

With multiple AI providers available (Ollama local, OpenAI cloud), the platform must select the appropriate provider for each inference request. Selection criteria include model name, required capabilities, user preference, and provider availability.

DECISION

Implement a **Provider Router** (`ProviderRouter` interface) with a deterministic 4-tier selection strategy:

1. **Model prefix routing:** `openai:gpt-4o` routes to the OpenAI provider, `ollama:qwen2.5` routes to Ollama
2. **Capability-based selection:** If a specific capability is requested (e.g., `STREAMING`), the router selects a provider whose models support that capability, as defined in the `ModelCapabilityRegistry`
3. **Preferred provider:** If the caller specifies a preferred provider, it is selected if available
4. **First available fallback:** The first provider reporting `isAvailable() == true` is selected

If no provider is available, the router throws `IllegalStateException` with a clear message.

Business services call `router.routeChat(InferenceRequest)` — they request capabilities, not specific providers.

ALTERNATIVES CONSIDERED

- **Direct injection of a single provider:** Rejected. Would make multi-provider setups impossible and require code changes to switch providers.
- **Random/round-robin selection:** Rejected. Non-deterministic behavior is harder to debug and audit.
- **Cost/latency-aware routing:** Deferred to future evolution. The router architecture supports adding routing dimensions.

CONSEQUENCES

- **Deterministic routing:** Same input always produces same routing decision
- **Auditable:** Every routing decision is logged (`log.debug("Routed '{}' to provider '{}'` by model prefix", ...))
- **Extensible:** Adding a routing dimension means adding a strategy tier, not rewriting the router
- **Clear failure mode:** Unavailable providers are skipped with explicit fallback

TRADE-OFFS

- Static capability registry requires manual updates when new models are added
- No runtime performance tracking for latency/cost-based routing
- Model prefix convention (`provider:model`) is a convention, not enforced by types

FUTURE EVOLUTION

- Cost-aware routing using `ModelCapability.estimatedCostPer1kTokens()`
- Latency-aware routing using `ModelCapability.estimatedLatencyMs()`
- Region-aware routing for multi-region deployments
- Dynamic capability discovery from provider APIs

See also: [\[\[ADR-003\]\]](#), [\[\[ADR-005\]\]](#), [\[\[ADR-006\]\]](#)

ADR-5

ADR-005 — Prompt Registry for Versioned Prompt Management

STATUS

Accepted. Implemented in `platform-`

`ai/src/main/java/com/cogniterra/platform/ai/application/DefaultPromptRegistry.java` .

CONTEXT

AI prompts are a critical platform asset. Without versioning, prompt changes cannot be tracked, audited, or rolled back. Without a registry, prompts are scattered across Java string constants, making them impossible to discover or manage.

DECISION

Implement a **Prompt Registry** (`PromptRegistry` interface) that stores versioned prompt templates with metadata:

Feature	Implementation
Versioning	<code>PromptTemplate.id + PromptTemplate.version</code> → qualified ID "rag-answer/v1"
Categories	<code>PromptTemplate.Category</code> : RETRIEVAL, SUMMARIZATION, EXTRACTION, CLASSIFICATION, EVALUATION, REASONING, WORKFLOW, GRAPH, SEARCH, SYSTEM
Variables	<code>{{variable}}</code> template substitution via <code>render(Map<String, String>)</code>
Metadata	<code>expectedOutputType</code> , <code>supportedModels</code> , <code>recommendedTemperature</code> , <code>examples</code>
Discovery	<code>findByCategory(Category)</code> , <code>listPromptIds()</code> , <code>getLatest(String)</code>

The default registry (`DefaultPromptRegistry`) seeds 8 prompts across 6 categories. Prompts are registered programmatically; in production, they would be loaded from YAML/JSON resources.

Every inference records which prompt was used (`RetrievalOrchestrationResult.promptTemplateId()`), enabling full reproducibility and audit trails.

ALTERNATIVES CONSIDERED

- **Hardcoded string constants:** Rejected. No versioning, no discovery, no audit trail.
- **Database-backed prompt store:** Deferred. Adds infrastructure dependency. In-memory registry with resource loading is sufficient for a reference implementation.
- **External prompt management service:** Rejected. Over-engineered for a modular monolith.

CONSEQUENCES

- **Reproducibility:** Every inference records `promptTemplateId` + `promptTemplateVersion`
- **Discoverability:** `findByCategory()` enables prompt inventory
- **Safe evolution:** New prompt versions can be registered without deleting old ones
- **Regression testing:** Prompts are identifiable assets that can be tested

TRADE-OFFS

- In-memory storage means prompts reset on restart (acceptable for a reference implementation)
- No prompt validation at registration time (variables are not checked against template)
- Rendering uses simple string substitution, not a template engine (deliberate simplicity)

FUTURE EVOLUTION

- YAML/JSON resource loading for external prompt definitions
- Prompt validation: verify all declared variables are used and all template variables are declared
- Prompt A/B testing: serve different versions to different users
- Prompt migration tooling for bulk updates

See also: [\[\[ADR-003\]\]](#), [\[\[ADR-004\]\]](#), [\[\[ADR-012\]\]](#)

ADR-6

ADR-006 — Model Capability Registry

STATUS

Accepted. Implemented in `platform-ai/src/main/java/com/cogniterra/platform/ai/application/DefaultModelCapabilityRegistry.java` .

CONTEXT

Different AI models have different capabilities (streaming, vision, JSON mode, tool calling, embeddings). Business logic should not hardcode assumptions like "model X supports JSON." Instead, capabilities should be queryable through a central registry, enabling the `ProviderRouter` to make intelligent selection decisions.

DECISION

Implement a **Model Capability Registry** (`ModelCapabilityRegistry` interface) that describes every available model's capabilities:

Capability	Examples
<code>supportsStreaming</code>	qwen2.5:7b (true), nomic-embed-text (false)
<code>supportsVision</code>	gpt-4o (true)
<code>supportsJson</code> / <code>supportsToolCalling</code>	gpt-4o, llama3.2 (true)
<code>supportsEmbeddings</code>	nomic-embed-text (true)
<code>supportsStructuredOutput</code>	gpt-4o (true)
<code>maxContextWindow</code>	128K (gpt-4o), 32K (qwen2.5)
<code>maxOutputTokens</code>	16384 (gpt-4o)
<code>estimatedLatencyMs</code>	500 (qwen2.5 local), 1200 (gpt-4o cloud)
<code>estimatedCostPer1kTokens</code>	\$5.00 (gpt-4o), \$0.00 (local Ollama)
<code>preferredUseCases</code>	"rag", "analysis", "summarization"

The registry supports querying by:

- Exact model name: `get("gpt-4o")`
- Provider: `findByProvider("ollama")`
- Capability: `findByCapability(EMBEDDING)`
- Provider + capability: `findByProviderAndCapability("ollama", STREAMING)`

The `DefaultModelCapabilityRegistry` seeds with 6 known models (4 Ollama, 2 OpenAI). The `ProviderRouter` uses the registry for capability-based routing decisions.

ALTERNATIVES CONSIDERED

- **Hardcoded if/else chains:** Rejected. Unmaintainable with growing model lists.
- **Runtime model discovery via API:** Deferred. Provider APIs (Ollama `/api/tags` , OpenAI `/models`) could populate the registry dynamically, but add latency and failure modes.
- **No registry — trust the provider:** Rejected. The platform needs to know capabilities before calling a model (e.g., "does this model support JSON mode?").

CONSEQUENCES

- **Capability-aware routing:** `ProviderRouter` selects models based on required capabilities
- **Static seed data:** Registry is populated at startup with known models; new models require code changes
- **Query interface:** Rich query API enables future use cases (e.g., "find the cheapest model that supports vision")

TRADE-OFFS

- Static data can become stale as providers add new models
- Estimated costs and latencies are approximations, not real-time measurements
- No runtime validation that the model actually supports claimed capabilities

FUTURE EVOLUTION

- Dynamic discovery from provider `/models` endpoints
- Runtime capability verification (send test prompts to verify claims)

- User-provided model registrations via configuration
- Integration with provider cost APIs for real-time pricing

See also: [[ADR-003]], [[ADR-004]], [[ADR-005]]

ADR-7

ADR-007 — Semantic Enrichment Engine

STATUS

Accepted. Implemented in `platform-`

`ai/src/main/java/com/cognitera/platform/ai/application/DefaultEnrichmentService.java` and `platform-api/src/main/java/com/cognitera/platform/api/ingestion/EnrichmentHook.java` .

CONTEXT

Documents contain unstructured information. To enable intelligent retrieval, the platform must extract structured knowledge — entities, concepts, and relationships — automatically during ingestion. Users should never manually populate knowledge bases.

DECISION

Implement an automatic **Semantic Enrichment Engine** that runs as part of the document ingestion pipeline:

```
Upload → Text Extraction → Enrichment → Chunking → Embedding → PostgreSQL + Qdrant
                        ↓
                    Neo4j Graph
```

The enrichment pipeline uses a dual-mode extraction strategy:

1. **LLM-based**: When a `ChatCompletionProvider` is available, prompts the LLM with `entity-extraction/v1` template for high-quality structured extraction
2. **Regex fallback**: When no LLM is available, uses regex patterns for known entity types (ORGANIZATION, PERSON, DATE, MONEY)

Results are bridged to Neo4j via `EnrichmentHook` , which converts `EnrichmentContext` to `GraphNode` / `GraphRelationship` objects and persists them through `GraphEnrichmentService` .

Entities carry **provenance**: `sourceDocumentId` , `chunkId` , `extractionConfidence` , `extractionTimestamp` , `extractionModel` , `promptVersion` , `provider` .

ALTERNATIVES CONSIDERED

- **Manual knowledge entry (old platform-knowledge module)**: Rejected. Users should not manually curate knowledge. The document corpus is the knowledge source. The old `platform-knowledge` module was removed in Phase 1.
- **LLM-only enrichment**: Rejected. Would fail when no LLM is available. Regex fallback ensures basic enrichment always works.
- **No enrichment**: Rejected. Without enrichment, retrieval is purely lexical/semantic with no structured understanding.

CONSEQUENCES

- **Automatic knowledge extraction**: Entities, concepts, and relationships are extracted during ingestion without user intervention
- **Graceful degradation**: Regex fallback when LLM is unavailable
- **Provenance**: Every graph node links back to its source document and extraction method
- **Neo4j population**: The knowledge graph is auto-generated, never manually curated

TRADE-OFFS

- Regex patterns are less accurate than LLM extraction
- The regex patterns are English-centric and need extension for multilingual documents

- Enrichment adds latency to the ingestion pipeline

FUTURE EVOLUTION

- Multilingual entity extraction patterns
- Domain-specific enrichment via `DomainConfiguration`
- Confidence threshold configuration for entity filtering
- Parallel enrichment for large documents
- Integration with external NER services

See also: [\[\[ADR-008\]\]](#), [\[\[ADR-009\]\]](#), [\[\[ADR-017\]\]](#), [\[\[ADR-018\]\]](#)

ADR-8

ADR-008 — GraphRAG — Graph-Enhanced Retrieval

STATUS

Accepted. Implemented in `Neo4jGraphSearchAdapter` bridging `GraphEnrichmentService` to `GraphSearchProvider`, with integration in `DefaultHybridRetrievalService`.

CONTEXT

Traditional RAG retrieves chunks via keyword and vector similarity. However, semantically related documents may not share keywords or embedding proximity. A knowledge graph connecting entities, concepts, and documents enables traversal-based discovery that complements keyword and vector retrieval.

DECISION

Implement **GraphRAG** — graph-enhanced retrieval — as an optional third retrieval source alongside keyword and vector:

```
SearchQuery
├─ KeywordSearchProvider.search() → keywordResults
├─ VectorSearchProvider.search() → vectorResults
├─ GraphSearchProvider.search() → graphResults (optional)
├─ merge(keyword, vector, graph) → fused candidates
│   ├── combine(): weighted linear fusion ( $k \times 0.40 + v \times 0.40 + c \times 0.20$ )
│   └─ combineWithGraph(): graph boost (existing +  $\text{graph} \times 0.15$ )
→ Reranking → LLM
```

Graph results **boost** existing candidates rather than replacing them. The `combineWithGraph()` method adds up to 15% score increase when graph traversal finds related nodes.

Graph retrieval is **optional**: `SearchMode.GRAPH` and `SearchMode.HYBRID_GRAPH` activate it. When Neo4j is unavailable, `NoOpGraphSearchProvider` returns empty results and retrieval continues with keyword + vector.

ALTERNATIVES CONSIDERED

- **Graph-only retrieval**: Rejected. Graph traversal is useful for discovery but less precise for exact match queries.
- **Graph as primary source with keyword/vector as secondary**: Rejected. Documents without graph entities would be invisible.
- **No graph retrieval**: Rejected. The enrichment engine already populates Neo4j; not using it for retrieval wastes the enrichment investment.

CONSEQUENCES

- **Three-source fusion**: Keyword, vector, and graph results are merged in a single pipeline
- **Graceful degradation**: Graph retrieval is optional; platform works without Neo4j
- **Graph boost**: Related entities and concepts boost document scores by up to 15%
- **Explainability**: Graph participation is tracked in retrieval metadata

TRADE-OFFS

- Graph search uses simple entity name matching from the query (not embedded query → nearest neighbor in graph embedding space)

- The `Neo4jGraphSearchAdapter` creates synthetic `ChunkReference` objects for graph results, which have lower fidelity than real chunk references
- Graph boost of 15% is fixed, not calibrated per domain

FUTURE EVOLUTION

- Graph embedding models for semantic graph traversal
- Configurable graph boost factors per domain
- Multi-hop reasoning via graph traversal patterns
- Graph-native reranking using centrality metrics

See also: [\[\[ADR-007\]\]](#), [\[\[ADR-009\]\]](#), [\[\[ADR-011\]\]](#)

ADR-9

ADR-009 — Neo4j as Knowledge Graph Persistence

STATUS

Accepted. Implemented in `platform-neo4j/src/main/java/com/cognitera/platform/neo4j/service/GraphEnrichmentService.java`.

CONTEXT

The platform needs to store structured knowledge extracted from documents — entities, concepts, and their relationships. This data is inherently graph-shaped: entities relate to documents, concepts relate to entities, documents cite other documents. A relational database would require complex recursive CTEs for graph traversal.

DECISION

Use **Neo4j** as the persistence layer for the automatically-generated knowledge graph. Neo4j is **not** a general-purpose database in this architecture — it stores only semantic enrichment output:

Node Type	Example
DOCUMENT	Uploaded PDF, DOCX, TXT
ENTITY	Organization, Person, Technology
CONCEPT	Temporal concepts, financial amounts, topics

Relationship	Example
MENTIONS	Document → Entity
RELATED_TO	Document → Concept
BELONGS_TO	Entity → Concept
REFERENCES	Document → Document

The graph is **auto-generated during ingestion** — never manually curated. The old `platform-knowledge` module (manual CRUD knowledge base) was removed in Phase 1.

Neo4j is **optional**: the `graph` profile in `docker-compose.yml` and `@ConditionalOnProperty(name = "platform.neo4j.uri")` ensure the platform starts without it.

ALTERNATIVES CONSIDERED

- **PostgreSQL with recursive CTEs**: Rejected. Graph traversal queries become complex and slow beyond 2-3 hops. Cypher is purpose-built for graph traversal.
- **Property graph in application memory**: Rejected. Does not persist across restarts; cannot scale beyond small corpora.

- **RDF triple store:** Rejected. Adds complexity (SPARQL, ontology management) without clear benefit over labeled property graphs for this use case.
- **No graph database:** Rejected. The enrichment engine produces graph-shaped data; storing it relationally would violate the "right tool for the data shape" principle.

CONSEQUENCES

- **Native graph traversal:** `GraphEnrichmentService.traverse(seedIds, maxDepth)` uses Cypher for multi-hop queries
- **Optional infrastructure:** Platform works without Neo4j
- **Provenance:** Every node carries `NodeProvenance` linking back to source document and extraction method
- **Auto-generated:** Graph population happens during ingestion, not through user interaction

TRADE-OFFS

- Adds infrastructure dependency when GraphRAG is desired
- Neo4j Community Edition has single-database limitation
- Graph schema is implicit (defined by code) rather than explicit (defined by constraints)

FUTURE EVOLUTION

- Graph embedding generation in Neo4j (GDS library)
- Cypher query templates for common traversal patterns
- Graph-native reranking using PageRank or centrality algorithms
- Multi-tenancy via Neo4j database-per-tenant (requires Enterprise)

See also: [\[\[ADR-007\]\]](#), [\[\[ADR-008\]\]](#), [\[\[ADR-018\]\]](#)

ADR-10

ADR-010 — Qdrant as Vector Database

STATUS

Accepted. Implemented in `platform-`

`search/src/main/java/com/cognitara/platform/search/application/qdrant/QdrantVectorSearchProvider.java` .

CONTEXT

Vector search requires a database optimized for high-dimensional similarity queries. General-purpose databases (PostgreSQL) can support vectors via extensions (pgvector) but are not optimized for vector-first workloads.

DECISION

Use **Qdrant** as the dedicated vector database. Qdrant is purpose-built for vector similarity search with:

- Native cosine similarity scoring
- Payload filtering (metadata alongside vectors)
- Collection management with configurable vector dimensions
- REST + gRPC APIs

The `QdrantVectorSearchProvider` communicates via REST API (`/collections/{name}/points/search`). Each vector point carries a payload with `chunkId` , `documentId` , `title` , `documentType` , `category` , `tags` , and `source` .

Qdrant is **optional**: when `platform.search.qdrant.host` is not configured, `NoOpVectorSearchProvider` provides empty search results and keyword search continues.

Configuration is managed via `QdrantProperties` (`@ConfigurationProperties(prefix = "platform.search.qdrant")`). The default collection is `document_intelligence_chunks` with 768-dimensional vectors (matching `nomic-embed-text` output).

ALTERNATIVES CONSIDERED

- **pgvector (PostgreSQL extension):** Rejected as primary vector store. While pgvector works for small-to-medium corpora, Qdrant provides better performance at scale, native quantization, and is purpose-built for vector search. pgvector is used for development/testing via H2 compatibility.

- **Weaviate, Pinecone, Milvus:** Rejected. Qdrant was chosen for its Rust performance, simple deployment (single binary), and REST API. The `VectorSearchProvider` SPI makes switching straightforward.
- **In-memory vector search:** Rejected. Does not persist across restarts.

CONSEQUENCES

- **High-performance vector search:** Cosine similarity with payload filtering
- **Collection auto-creation:** `QdrantCollectionManager.ensureCollectionExists()` on startup
- **Batch indexing:** `indexBatch()` for efficient bulk ingestion
- **Graceful degradation:** Platform works without Qdrant (keyword-only search)

TRADE-OFFS

- Additional infrastructure dependency in production
- REST API adds ~5-10ms latency vs gRPC
- No built-in quantization or disk-based indexing in the current configuration

FUTURE EVOLUTION

- gRPC client for lower latency
- Qdrant quantization for memory efficiency with large corpora
- Multi-collection strategy per tenant or document type
- Hybrid search pushdown to Qdrant (if Qdrant adds text search)

See also: [\[\[ADR-008\]\]](#), [\[\[ADR-011\]\]](#)

ADR-11

ADR-011 — Retrieval Orchestration with Intent-Based Strategy Selection

STATUS

Accepted. Implemented in `platform-ai/src/main/java/com/cognitera/platform/ai/application/DefaultRetrievalOrchestrator.java`.

CONTEXT

Different query types benefit from different retrieval strategies. A factual lookup ("What was the revenue in Q2?") benefits from keyword search. An exploratory question ("What capabilities does the platform have?") benefits from hybrid search. An index inspection query should not waste resources on vector search.

DECISION

Implement a **Retrieval Orchestrator** that selects the retrieval strategy based on classified query intent:

User Query → Intent Classification → Strategy Selection → Search Execution → Result

Strategy mapping (deterministic and explainable):

QueryIntent	SearchMode	Rationale
INDEX_INSPECTION, CORPUS_DISCOVERY	KEYWORD	Exact match queries; vector search unnecessary
QUESTION_ANSWERING, WORKSPACE_ANALYSIS, SOURCE_ANALYSIS	HYBRID	Complex queries benefit from keyword + vector fusion
DOCUMENT_RESEARCH, DOCUMENT_LOOKUP	SEMANTIC	Conceptual queries benefit from vector similarity

The orchestrator produces `RetrievalOrchestrationResult` with full **explainability metadata**: intent, strategy, mode, prompt template, result counts, fusion method, reranking status, timing, and a step-by-step `traceLog`.

ALTERNATIVES CONSIDERED

- **Always run all retrievers:** Rejected. Wastes resources on inappropriate strategies (e.g., vector search for exact ID lookups).

- **User-specified strategy:** Rejected. Users should not need to understand retrieval internals.
- **ML-based strategy selection:** Deferred. A trained classifier could outperform keyword-based intent classification, but keyword rules are deterministic, explainable, and sufficient for initial release.

CONSEQUENCES

- **Deterministic:** Same query always produces same strategy
- **Explainable:** Every retrieval decision is recorded in `traceLog`
- **Efficient:** Only appropriate retrievers are executed
- **Prompt-aware:** Retrieves the latest prompt template from `PromptRegistry`

TRADE-OFFS

- Keyword-based intent classification can misclassify queries
- Strategy mapping is static (no runtime adaptation based on result quality)
- Graph retrieval is not automatically selected by the orchestrator (requires explicit `HYBRID_GRAPH` mode)

FUTURE EVOLUTION

- ML-based intent classification for higher accuracy
- Feedback loop: if results are poor, retry with expanded strategy
- Graph strategy auto-selection when enriched entities are detected in the query

See also: `[[ADR-005]]`, `[[ADR-008]]`, `[[ADR-012]]`

ADR-12

ADR-012 — Explainability by Default

STATUS

Accepted. Implemented in `platform-ai/src/main/java/com/cognitera/platform/ai/model/RetrievalOrchestrationResult.java` and `platform-ai/src/main/java/com/cognitera/platform/ai/model/InferenceMetadata.java`.

CONTEXT

AI systems must be auditable. When the platform generates an answer, stakeholders need to know: which model generated it, which prompt template was used, which documents were retrieved, which retrieval strategy was selected, and how long each step took. Without explainability, AI output is a black box.

DECISION

Make **explainability a first-class concern** on every inference. Every AI response carries `InferenceMetadata` and every retrieval carries `RetrievalOrchestrationResult` with:

Metadata	Example
<code>provider</code>	"ollama"
<code>model</code>	"qwen2.5:14b"
<code>promptTemplateId</code>	"rag-answer/v1"
<code>retrievalStrategy</code>	"HYBRID"
<code>keywordResultCount</code>	12
<code>vectorResultCount</code>	8
<code>graphNodeCount</code>	3
<code>totalSourceCount</code>	15
<code>fusionMethod</code>	"weighted-linear-fusion"

rerankingApplied	true
rerankingProvider	"ollama-cross-encoder"
retrievalStartedAt / retrievalCompletedAt	timestamps
traceLog	["Orchestration started", "Intent: QUESTION_ANSWERING", ...]
evaluationScores	{grounding: 0.72, faithfulness: 0.85}

The `explain()` method produces a human-readable summary:

```
Intent: QUESTION_ANSWERING | Strategy: HYBRID | Prompt: rag-answer/v1 | Sources: 15 | Fusion:
weighted-linear-fusion | Reranking: yes (ollama-cross-encoder) | Duration: 245ms
```

Explainability is **built into the orchestration layer**, not bolted on as an afterthought. Business services don't call explainability APIs — the orchestrator populates metadata automatically.

ALTERNATIVES CONSIDERED

- **Optional explainability:** Rejected. Explainability should never be optional in an enterprise AI platform.
- **Separate explainability service:** Rejected. Would require duplicating orchestration state. Building metadata into the orchestration result ensures consistency.
- **User-facing explainability only:** Rejected. Internal diagnostics are as important as user-facing explanations.

CONSEQUENCES

- **Every inference is auditable:** Full traceability from query to answer
- **Deterministic:** Same query → same strategy → same explainability output
- **Low overhead:** Metadata is collected during normal execution, not as a separate pass
- **Future-proof:** New retrieval sources automatically appear in metadata

TRADE-OFFS

- `traceLog` is append-only string list (not structured log entries)
- `RetrievalOrchestrationResult` uses a builder with 19 setters (verbose but explicit)
- Metadata is not yet exposed through a dedicated API endpoint

FUTURE EVOLUTION

- Structured trace events with typed metadata (not string list)
- Explainability REST API (`GET /api/inferences/{id}/explain`)
- Visualization of retrieval decisions (Sankey diagram of query → strategy → results)
- Differential explainability: "why was document A ranked above document B?"

See also: [\[\[ADR-011\]\]](#), [\[\[ADR-013\]\]](#), [\[\[ADR-014\]\]](#)

ADR-13

ADR-013 — Evaluation Engine

STATUS

Accepted. Implemented in `platform-ai/src/main/java/com/cogniterra/platform/ai/application/DefaultEvaluationService.java` .

CONTEXT

AI-generated answers must be evaluated for quality. Without evaluation, there is no feedback loop to detect hallucination, poor grounding, or irrelevant answers. Evaluation must run automatically as part of the inference pipeline, not as a separate manual process.

DECISION

Implement an **Evaluation Engine** that automatically evaluates every retrieval + inference cycle:

Metric	Method	Range
groundingScore	Context length vs answer length ratio	[0, 1]
citationCoverage	Citation markers [1], [2] in answer	[0, 1]
faithfulness	Inverse of hallucination indicators	[0, 1]
answerRelevance	Term overlap between question and answer	[0, 1]
contextRelevance	Term overlap between question and context	[0, 1]
hallucinationIndicators	Uncertainty phrase detection	integer ≥ 0
passed	Composite quality gate	boolean

The `DefaultEvaluationService` uses heuristic methods (regex patterns, term overlap, length ratios). In production, a dedicated evaluation LLM or framework (deepeval, ragas) would replace these heuristics.

Evaluation is wired into `DefaultRetrievalOrchestrator` — it runs after retrieval and before the result is returned. Every `RetrievalOrchestrationResult` carries evaluation scores in its metadata.

ALTERNATIVES CONSIDERED

- **No evaluation:** Rejected. An AI platform without quality feedback is irresponsible engineering.
- **LLM-as-judge only:** Deferred. Requires a second LLM call, adding latency and cost. Heuristic evaluation provides fast, deterministic feedback. LLM evaluation can be added as a separate evaluation profile.
- **Human evaluation only:** Rejected. Does not scale and cannot run in CI pipelines.

CONSEQUENCES

- **Automatic quality gate:** Every retrieval is scored; low-quality retrievals are flagged
- **Deterministic:** Heuristic methods produce consistent, reproducible scores
- **Lightweight:** Evaluation adds negligible latency (no LLM calls)

TRADE-OFFS

- Heuristic methods are less accurate than LLM-based evaluation
- `hallucinationIndicators` uses regex patterns that miss sophisticated hallucinations
- No benchmark dataset integration for regression testing

FUTURE EVOLUTION

- LLM-as-judge evaluation (separate profile, gated by configuration)
- RAG evaluation framework integration (deepeval, ragas)
- Evaluation benchmark dataset with ground truth annotations
- Automated regression testing: "does this prompt change improve or degrade evaluation scores?"

See also: [\[\[ADR-011\]\]](#), [\[\[ADR-012\]\]](#), [\[\[ADR-014\]\]](#)

ADR-14

ADR-014 — Workflow Engine

STATUS

Accepted. Implemented in `platform-ai/src/main/java/com/cogniterra/platform/ai/application/DefaultWorkflowEngine.java`.

CONTEXT

Enterprise AI applications involve multi-step processes: document upload → extraction → enrichment → analysis → review → completion. These processes have defined steps, transitions, and state. Hardcoding step logic in controllers couples workflow to HTTP concerns. A reusable workflow engine separates process definition from execution.

DECISION

Implement a reusable **Workflow Engine** (`WorkflowEngine` interface):

Concept	Implementation
<code>WorkflowDefinition</code>	Named workflow with ordered steps and transitions
<code>WorkflowStep</code>	Step with id, name, description, handler type (manual/automated/ai)
<code>WorkflowInstance</code>	Running instance with current step, status, context

Two pre-registered workflows demonstrate the engine:

1. **document-intelligence** : SETUP → INGESTION → ANALYSIS → REVIEW → COMPLETE (5 steps)
2. **batch-ingestion** : INGEST → ENRICH → COMPLETE (3 steps)

The engine supports:

- `start(definitionId, context)` — creates a new instance at the initial step
- `advance(instanceId)` — moves to the next step; marks COMPLETED when no further steps
- `previous(instanceId)` — returns to the previous step
- `findInstance(instanceId)` — retrieves instance state
- `listActive()` — finds all active instances

The current implementation is in-memory (suitable for single-node). A database-backed store would be needed for multi-node deployments.

ALTERNATIVES CONSIDERED

- **Hardcoded wizard in controller:** Rejected. The old workspace wizard was a hardcoded switch statement in `WorkspacePageController` . A reusable engine separates process logic from presentation.
- **Camunda/Flowable BPMN engine:** Rejected. Over-engineered for the current use case. The lightweight engine demonstrates the concept without framework lock-in.
- **Spring State Machine:** Rejected. Adds framework dependency for a relatively simple state transition model.

CONSEQUENCES

- **Reusable:** Any module can define and execute workflows
- **Simple API:** 5 methods, intuitive lifecycle
- **Pre-registered workflows:** Demonstrates real usage without configuration files

TRADE-OFFS

- In-memory storage means workflows reset on restart
- No persistence or fault tolerance for long-running workflows
- No parallel step execution or conditional branching
- Step handler types are declarative (manual/automated/ai) but not enforced

FUTURE EVOLUTION

- Database-backed instance storage for production durability
- Conditional transitions based on step outcomes
- Timer-based transitions (escalation after timeout)
- Visual workflow definition (BPMN subset)
- Workflow event publishing for audit integration

See also: [\[\[ADR-001\]\]](#), [\[\[ADR-020\]\]](#)

ADR-15

ADR-015 — AI Observability with Micrometer
STATUS

Accepted. Implemented in `platform-observability/src/main/java/com/cogniterra/platform/observability/metrics/AiMetrics.java` .

CONTEXT

AI operations are complex, multi-step processes with multiple external dependencies. Without observability, diagnosing failures (is the LLM slow? is Qdrant down? is enrichment producing too few entities?) requires log diving. Production AI systems require metrics.

DECISION

Implement **AI-specific observability** using Micrometer + Prometheus, integrated into a dedicated `platform-observability` module:

Metric	Type	Tags
ai.inference.duration	Timer (percentile histogram)	provider, model
ai.embedding.duration	Timer	provider, model
ai.retrieval.duration	Timer	mode (keyword/semantic/hybrid)
ai.graph.retrieval.duration	Timer	—
ai.enrichment.duration	Timer	—
ai.ingestion.duration	Timer	document_type
ai.prompt.duration	Timer	template
ai.provider.available	Gauge (0/1)	provider
ai.embedding.count	Counter	provider
ai.enrichment.entities	Counter	—
ai.evaluation.*	Summary	metric name

Metrics are exposed via `/actuator/prometheus` and `/actuator/health`. Health indicators check Ollama, Qdrant, and provider availability.

The `platform-observability` module is reusable across future applications. It depends only on Micrometer and Spring Boot Actuator — no platform-specific dependencies.

ALTERNATIVES CONSIDERED

- **Logging only:** Rejected. Logs are not aggregatable or queryable at scale. Metrics enable dashboards and alerting.
- **OpenTelemetry from day one:** Deferred. Micrometer provides a simpler API that can export to OTLP when OpenTelemetry is adopted.
- **Metrics in each module:** Rejected. Centralizing metrics in `platform-observability` ensures consistent naming, avoids duplication, and enables reuse.

CONSEQUENCES

- **Operational visibility:** Every AI operation produces metrics
- **Reusable module:** Future applications get observability by depending on `platform-observability`
- **Standard format:** Prometheus exposition format enables Grafana dashboards
- **Health endpoints:** Liveness, readiness, and dependency health checks via Actuator

TRADE-OFFS

- Metrics are defined but not yet fully wired into all services (`AiMetrics` is injected in fewer places than ideal)
- No custom Grafana dashboard definitions
- No alerting rules (would be added in deployment configuration, not in the platform)

FUTURE EVOLUTION

- Wire `AiMetrics` into all AI services (`AiService` , `SearchService` , `DefaultEnrichmentService`)
- OpenTelemetry exporter for distributed tracing

- Pre-built Grafana dashboard JSON
- Alert definitions for critical metrics (LLM latency > threshold, provider down)

See also: [[ADR-002]], [[ADR-016]]

ADR-16

ADR-016 — Graceful Degradation Over Hard Failures

STATUS

Accepted. Implemented throughout the platform via `@ConditionalOnProperty`, `@ConditionalOnBean`, `ObjectProvider`, and `NoOp*` fallback beans.

CONTEXT

The Enterprise AI Platform depends on multiple external services: Ollama (LLM), Qdrant (vector DB), Neo4j (graph DB), PostgreSQL (relational DB). In production, any of these may be unavailable. The platform must continue functioning with reduced capabilities rather than failing entirely.

DECISION

Design every external dependency as **optional**. The platform uses multiple Spring mechanisms for graceful degradation:

Mechanism	Example	Behavior
<code>@ConditionalOnProperty</code>	<code>OllamaChatProvider</code> requires <code>platform.ai.ollama.base-url</code>	Bean not created if config absent
<code>@ConditionalOnBean</code>	<code>Neo4jGraphSearchAdapter</code> requires <code>GraphEnrichmentService</code>	Bean not created if Neo4j unavailable
<code>ObjectProvider<T></code>	<code>DefaultDocumentIngestionProcessor</code> injects <code>ObjectProvider<EnrichmentHook></code>	Null-safe <code>getIfAvailable()</code>
<code>NoOp*</code> fallbacks	<code>NoOpVectorSearchProvider</code> , <code>NoOpGraphSearchProvider</code>	Return empty results, never throw
try-catch wrappers	<code>Neo4jGraphSearchAdapter.search()</code>	Log warning, return empty list
<code>isAvailable()</code> checks	<code>GraphSearchProvider.isAvailable()</code> , <code>ChatCompletionProvider.isAvailable()</code>	Caller checks before invoking
<code>ProviderRouter</code> fallback	4-tier selection skips unavailable providers	Throws only when ALL unavailable

The degradation is **visible**: health indicators report status, metrics track failures, logs record the reason. The platform never silently degrades.

ALTERNATIVES CONSIDERED

- **Mandatory all services**: Rejected. Would prevent development without full infrastructure.
- **Circuit breaker only**: Rejected. Circuit breakers (Resilience4j) are valuable for transient failures but don't address the "service never configured" case. Conditional beans handle both.
- **Feature flags**: Rejected. Adds complexity. Conditional bean activation is simpler and more idiomatic in Spring.

CONSEQUENCES

- **Platform starts with zero infrastructure** (except PostgreSQL for basic operation)
- **157 tests pass without any external services** (H2 in-memory, no Ollama/Qdrant/Neo4j)
- **Clear failure modes**: When a service is down, the reason is logged and surfaced in health endpoints

- **Gradual capability ramp:** Start with keyword search → add Qdrant for vector → add Neo4j for GraphRAG → add Ollama for LLM

TRADE-OFFS

- Some operations silently return empty results (graph search) rather than surfacing errors to users
- No retry logic for transient failures (deferred to Resilience4j integration)
- Health indicators currently read env vars directly rather than Spring config (known drift, documented)

FUTURE EVOLUTION

- Resilience4j integration for retry, circuit breaker, rate limiter
- Health indicators refactored to inject `@ConfigurationProperties`
- Degradation event publishing to audit log
- User-facing capability indicators ("vector search unavailable")

See also: [\[\[ADR-003\]\]](#), [\[\[ADR-008\]\]](#), [\[\[ADR-009\]\]](#), [\[\[ADR-010\]\]](#), [\[\[ADR-015\]\]](#)

ADR-17

ADR-017 — DomainConfiguration for Multi-Domain AI

STATUS

Accepted. Interface defined in `platform-ai/src/main/java/com/cognitera/platform/ai/api/DomainConfiguration.java`. Default implementations embedded in existing services.

CONTEXT

The platform must support multiple application domains (contract intelligence, financial analysis, regulatory compliance, technical documentation) from a single codebase. Domain-specific logic (concept definitions, analysis objectives, finding hierarchies, system instructions) must not be hardcoded in platform services. The platform core must remain domain-independent.

DECISION

Define a **DomainConfiguration SPI** that encapsulates domain-specific AI behavior:

Method	Returns	Purpose
<code>domainId() / displayName()</code>	String	Identity
<code>concepts()</code>	List<ConceptDefinition>	Keywords + governing references per concept
<code>objectives()</code>	List<ObjectiveDefinition>	Analysis objectives with keywords
<code>findingRoleMapping()</code>	Map<String, String>	Reference → finding role mapping
<code>findingRelationships()</code>	List<FindingRelationship>	Relationships between findings
<code>centralityWeights()</code>	Map<String, Double>	Centrality weights for references
<code>peripheralReferences()</code>	Set<String>	References classified as peripheral
<code>roleKeywords()</code>	Map<String, List<String>>	Source role classification keywords
<code>systemInstruction()</code>	String	Domain-specific AI system instruction
<code>answerStructureGuidance()</code>	String	Domain-specific answer structure

Applications provide their `DomainConfiguration` as a Spring bean. The platform's existing services (`DefaultConceptExtractionService`, `DefaultObjectiveAnalysisService`, etc.) currently embed default configurations. These defaults represent the original document intelligence domain but should be migrated to `DomainConfiguration` implementations.

ALTERNATIVES CONSIDERED

- **Hardcode domain logic in each service:** Rejected. Couples the platform to a single domain and prevents reuse.
- **Database-driven configuration:** Deferred. Adds infrastructure dependency. SPI-based configuration is simpler and testable.
- **Remove all domain logic from platform:** Rejected. The platform must demonstrate real AI behavior. DomainConfiguration keeps domain logic while making it swappable.

CONSEQUENCES

- **Extension point exists:** New domains implement DomainConfiguration without touching platform code
- **Migration path:** Existing hardcoded domain rules in 8 services can migrate to DomainConfiguration implementations incrementally
- **Testability:** Domain configurations can be tested independently

TRADE-OFFS

- Currently a forward-looking SPI — the 8 existing services still embed their configurations rather than consuming DomainConfiguration
- No multi-domain routing (which DomainConfiguration to use for a given query)
- No domain discovery mechanism

FUTURE EVOLUTION

- Migrate 8 services to consume DomainConfiguration
- Multi-domain routing based on query classification
- Domain configuration discovery (List<DomainConfiguration> injection for multi-domain setups)
- External domain configuration via YAML/JSON files

See also: [[ADR-003]], [[ADR-007]]

ADR-18

ADR-018 — Provenance-Aware Knowledge Graph

STATUS

Accepted. Implemented in platform-neo4j/src/main/java/com/cognitera/platform/neo4j/model/GraphNode.NodeProvenance .

CONTEXT

Knowledge graphs generated from AI extraction are only trustworthy if every node and edge can be traced back to its source. Without provenance, a graph node claiming "Acme Corporation is an ORGANIZATION" cannot be audited — was this extracted from a financial report or hallucinated by an LLM?

DECISION

Every graph node and relationship carries NodeProvenance :

Field	Purpose
sourceDocumentId	Which document was the source
chunkId	Which chunk within the document
chunkOffset	Position within the chunk
extractionConfidence	How confident was the extraction (0.0–1.0)
extractionTimestamp	When was it extracted
extractionModel	Which model performed the extraction
promptVersion	Which prompt template version was used
provider	Which AI provider performed the extraction

Provenance is captured during enrichment (`DefaultEnrichmentService`), bridged through `EnrichmentHook` , and persisted to Neo4j by `GraphEnrichmentService` . The data flows: `EnrichmentContext` → `EnrichmentResult` → `GraphNode` (with `NodeProvenance`) → Neo4j.

ALTERNATIVES CONSIDERED

- **No provenance**: Rejected. An unauditable knowledge graph is useless for enterprise applications.
- **Separate provenance table in PostgreSQL**: Rejected. Graph data should carry its own provenance. A separate store creates consistency challenges.
- **Blockchain-based provenance**: Rejected. Over-engineered. Immutable audit log in PostgreSQL + provenance in Neo4j provides sufficient traceability.

CONSEQUENCES

- **Full traceability**: Every graph element can be traced to its source document, chunk, model, prompt, and provider
- **Confidence-aware**: Extraction confidence enables filtering low-confidence extractions
- **Reproducibility**: Re-extraction with different models/prompts can be compared by timestamp and version

TRADE-OFFS

- Provenance adds storage overhead to every node and relationship
- `extractionConfidence` is self-reported by the extraction method (not independently verified)
- Provenance is not yet leveraged for retrieval scoring (e.g., boost high-confidence extractions)

FUTURE EVOLUTION

- Confidence-weighted retrieval: boost documents with high-confidence extractions
- Provenance visualization in the UI
- Automated re-extraction when prompt version changes
- Provenance chain: "entity A was extracted from chunk B of document C by model D using prompt E at time F"

See also: [\[\[ADR-007\]\]](#), [\[\[ADR-008\]\]](#), [\[\[ADR-009\]\]](#)

ADR-19

ADR-019 — Multi-Layer Testing Strategy

STATUS

Accepted. Implemented across `platform-ai/src/test/` and `platform-api/src/test/` . Documented in `TESTING.md` .

CONTEXT

An Enterprise AI Platform requires confidence that every architectural subsystem behaves correctly. Tests must validate behavior, not implementation details. A reader should understand the platform architecture by reading the tests.

DECISION

Adopt a **5-layer testing strategy**:

Layer	Location	Execution	Purpose
Unit	<code>platform-ai/src/test/.../unit/</code>	<code>mvn test</code>	Validate components in isolation
Integration	<code>platform-api/src/test/.../ai/</code>	<code>mvn verify</code>	Validate subsystems with Spring context
Architecture	<code>platform-api/src/test/.../architecture/</code>	<code>mvn verify</code>	Validate end-to-end behavior
Contract	<code>platform-ai/src/test/.../contract/</code>	<code>mvn verify</code>	Validate SPI stability
Playwright (UI)	<code>e2e-tests/playwright/</code>	<code>mvn verify -Pui-tests</code>	Validate browser behavior

Key principles:

- **Behavior over implementation:** Tests verify what the platform does, not how it does it
- **Executable documentation:** Test names describe behavior ("routes openai:gpt-4o to openai provider")
- **Realistic scenarios:** Architecture tests use a test-corpus/ with real document types and expected behaviors
- **Contract tests:** Every ChatCompletionProvider implementation must satisfy the abstract ChatCompletionProviderContract
- **No mocks for platform internals:** Unit tests mock external providers; integration tests use real Spring context with H2

157 tests validate: Prompt Registry (12), Model Capability Registry (12), Provider Router (14), Provider Contracts (5), Retrieval Orchestrator (3), Evaluation Engine (8), Workflow Engine (10), Semantic Enrichment (14), Graceful Degradation (9), GraphRAG (4), AI Pipeline (10), Browser UI (12), Platform (91).

ALTERNATIVES CONSIDERED

- **Coverage-driven testing:** Rejected. Coverage percentage does not measure architectural confidence. 157 behavioral tests provide more value than 500 getter tests.
- **BDD framework (Cucumber):** Rejected. Adds framework complexity. JUnit 5 @DisplayName + @Nested provides sufficient behavior description.
- **No UI tests:** Rejected. Browser tests validate user-visible behavior that unit tests cannot.

CONSEQUENCES

- **Architecture as tests:** Test structure mirrors platform architecture
- **Confidence, not coverage:** Every major subsystem has executable proof of correctness
- **CI-ready:** mvn clean verify runs all tests except Playwright (separate profile)
- **Contract protection:** New providers must satisfy SPI contracts

TRADE-OFFS

- Playwright tests require a running application (separate Maven profile)
- test-corpus/ is small (3 documents) — sufficient for architectural validation, not exhaustive
- No performance regression tests yet

FUTURE EVOLUTION

- Performance smoke test suite (mvn verify -Pperformance)
- Extended test corpus with multilingual documents
- Snapshot tests for prompt rendering output
- Automated evaluation regression: "did this prompt change degrade faithfulness?"

See also: [[ADR-020]], [TESTING.md](#)

ADR-20

ADR-020 — Enterprise AI Platform Design Philosophy

STATUS

Accepted. Embodied in every architectural decision across the platform.

CONTEXT

The platform is not a chatbot. It is not a RAG demo. It is a reference implementation of an Enterprise AI Platform designed to be reusable across multiple AI-powered applications. Every architectural decision flows from this philosophy.

DECISION

The platform is governed by these design principles:

1. AI is Infrastructure AI is not a feature. It is infrastructure, like databases or message queues. Business logic depends on abstract interfaces (ChatCompletionProvider , EmbeddingProvider), not concrete implementations (OllamaChatProvider , OpenAiChatProvider). Adding a new AI provider requires zero changes to business logic.

2. Modular Monolith Over Microservices Module boundaries are enforced at compile time. Clear dependency direction prevents cycles. The architecture supports future extraction into microservices but does not prematurely distribute.

3. Graceful Degradation is Mandatory Every external dependency is optional. The platform starts with zero infrastructure and gains capabilities as services become available. Keyword search works without Qdrant. Retrieval works without Neo4j. Inference works without Ollama.

4. Explainability by Default Every AI operation produces auditable metadata: which model, which prompt, which strategy, which documents, how long it took. Explainability is not a feature toggle — it is built into the orchestration layer.

5. Documents Are the Knowledge Source Knowledge is extracted from documents automatically during ingestion. There is no manual knowledge base, no CRUD interface for entities. The enrichment engine extracts entities, concepts, and relationships. The knowledge graph is auto-generated.

6. Domain Independence The platform core contains no domain-specific logic. Domain customization happens through `DomainConfiguration` implementations. The platform can serve contract intelligence, financial analysis, compliance, or technical documentation from the same codebase.

7. Testing as Architecture Documentation Tests validate behavior, not implementation. Test structure mirrors platform architecture. A Principal Engineer should understand the platform by reading the test suite.

8. Production Readiness Every subsystem considers: how does it fail? How is it observed? How is it configured? Conditional beans, health indicators, metrics, structured logging, and configuration properties are not afterthoughts — they are first-class concerns.

ALTERNATIVES CONSIDERED

The alternative philosophies considered and rejected:

- **"Move fast and break things"**: Incompatible with enterprise AI. Every decision is deliberate, documented, and testable.
- **"Maximize features, minimize architecture"**: Would produce a prototype, not a platform. The architecture is the product.
- **"AI is magic"**: Treating AI as opaque magic leads to unmaintainable systems. Every AI operation is instrumented, evaluated, and auditable.

CONSEQUENCES

- **Reusable platform**: Future applications (contract intelligence, financial analysis, compliance) can be built on the same foundation
- **Clear extension points**: `DomainConfiguration`, `ChatCompletionProvider`, `GraphSearchProvider` define where the platform grows
- **Professional engineering**: The codebase demonstrates architecture, testing, observability, and resilience — not just AI integration
- **9 modules, 157 tests, 0 failures**: Every architectural subsystem participates in real execution paths

TRADE-OFFS

- Architectural rigor means more interfaces and abstractions than a prototype would have
- The platform is not optimized for the shortest path to a demo — it is optimized for long-term maintainability
- Some SPLs (`DomainConfiguration`) are forward-looking and not yet consumed by all eligible services

FUTURE EVOLUTION

This document should remain stable. New ADRs should reference these principles. If a proposed change conflicts with a principle, it should either be rejected or this ADR should be amended with explicit rationale.

See also: [\[\[ADR-001\]\]](#), [\[\[ADR-003\]\]](#), [\[\[ADR-012\]\]](#), [\[\[ADR-016\]\]](#), [\[\[ADR-017\]\]](#), [\[\[ADR-019\]\]](#)
